Multi-agent LLM-based Framework for Code Clone Refactoring

ABSTRACT

Code clones are common in software systems and often lead to maintenance and bug-related issues. Extract Method is a standard refactoring technique to remove duplication, but applying it correctly requires preserving behavior, compiling successfully, and satisfying structural constraints. Recent studies show that single-prompt Large Language Models (LLMs) can generate syntactically correct refactorings, yet they frequently produce incomplete or structurally invalid transformations, especially for Type-2 and Type-3 clones.

In this paper, we present a multi-agent workflow for clone refactoring in an interactive development environment. The workflow decomposes the task into structured stages, including clone detection, refactoring generation, structural usefulness checking, compilation validation, and test-based behavioral verification. We also incorporate retrieval-augmented generation (RAG) to guide the refactoring agent with curated Extract Method examples, and we introduce an explicit usefulness checker to enforce structural correctness beyond compilation and testing.

We evaluate our approach on 60 manually constructed clone cases from the JHotDraw Java project, covering Type-1, Type-2, and Type-3 clones. We compare four modern LLMs (GPT-5.2, Gemini 3 Pro, Claude Opus 4.6, and Deepseek v3.2) under both multi-agent and single-LLM settings, and we further compare against the traditional refactoring tool JDeodorant. Results show that the multi-agent workflow improves overall end-to-end refactoring success from 71.3% in the single-LLM setting to 86.3%, mainly by reducing structurally incorrect extractions and enabling iterative repair through feedback. Ablation analysis further demonstrates that usefulness checking and feedback loops contribute more to reliability than detection alone.

Our findings suggest that reliable LLM-based refactoring requires structured decomposition, explicit structural validation, and iterative feedback, rather than relying on one-shot code generation.

CCS CONCEPTS

• **Software Engineering → Software Testing, Quality Assurance, and Maintenance**;

KEYWORDS

software quality, mining software repositories

1 INTRODUCTION

Nowadays there are a lot of code clones in software development as developers could save time by just copying and pasting the reused logic. Although this action can help speed up the development, it could also introduce bugs and issues for maintenance. For example, if the same code fragments exist in multiple places in the project, when developers make changes to them, those changes should be applied to every copy. Missing one copy could cause the inconsistency problems and make the project harder to maintain in the future [23, 37].

To solve the issues that cloned codes may bring, it is important to refactor those codes. Code refactoring refers to the process of improving structures without changing internal behavior [18]. There are many different code refactoring methods, among them, the Extract Method refactoring is one of the most used technique to address code clone issues [7, 18, 29]. Extract Method refactoring moves the cloned codes into a single method, and then replaces the original clones with shared method calls. However, when applying Extract Method refactoring, developers need to make sure it preserves the original functionalities and compiles without any issues, which make the refactoring not easy.

As Large Language Models (LLMs) emerged fast during these years, researchers have begun to study whether LLMs can assist with code clone detection [16, 43] and refactoring [12, 26]. Recent models from OpenAi and other providers can generate code and understand programming patterns. Recent empirical study [35] evaluated GPT-4o's ability to classify and refactor code clones in a real world project. Their study found that LLMs performed well on simple clones. However, when the clones became more complex, the performance dropped significantly. They also found several LLMs hallucinated patterns regarding Extract Method refactorings. The refactorings generated by LLMs do not always follow the definition of Extract Method, even though the models are explicitly instructed, so the extraction patterns are not always useful. In their work, they found patterns such as incomplete refactoring, directly deleted clones, fragmented logic into multiple small methods, or extracted more code than necessary. For example, Listing 1 shows one example from the empirical study [35]. In line 30, the refactoring proposed by LLM directly replaced the cloned method init_Cloned() with method call to the original method init().

Listing 1: Incorrect Extract Method refactoring example

```
1  public void init() {
2      createIconkit();
3      getVersionControlStrategy().assertCompatibleVersion();
4      setUndoManager(new UndoManager());
5
6      getContentPane().setLayout(new BorderLayout());
7
8      fView = createDrawingView();
9
10     JPanel attributes = createAttributesPanel();
11     createAttributeChoices(attributes);
12     getContentPane().add("North", attributes);
13
14     JPanel toolPanel = createToolPalette();
15     createTools(toolPanel);
16     getContentPane().add("West", toolPanel);
17
18     getContentPane().add("Center", (Component)view());
19     JPanel buttonPalette = createButtonPanel();
20     createButtons(buttonPalette);
21     getContentPane().add("South", buttonPalette);
22
23     initDrawing();
24     // JFC should have its own internal double buffering...
25     //setBufferedDisplayUpdate();
26     setupAttributes();
```

```
}

public void init_Cloned() {
      init();
}
```

These findings suggest that a single LLM cannot reliably handle the complete detection and correction. Refactoring requires multiple steps: detecting clones, planning the refactoring, generating code, checking compilation, and running tests. Multi-agent systems [9, 32, 38, 42] offer a new direction for clone detection and refactoring. Instead of using single LLM for the entire task, we can assign different roles to specialized agents. Each agent focuses on a specific task and uses tools to interact with the development environment. This design has been successful in other software engineering tasks. For example, RefAgent [32] uses four specialized agents for refactoring and achieves a 90% unit test pass rate through iterative feedback loops.

Inspired by these approaches, we propose a multi-agent framework specifically designed for code clone detection and refactoring. Our framework is triggered when a developer pastes code, inspired by the AntiCopyPaster tool [3–6].

Our framework addresses the limitations observed in previous work through several designs. Instead of relying on a single LLM to do both detection and refactoring, we adopt a multi-agent workflow. In our setting, we design several agents where each one is responsible for one specific task.

First, we decompose the refactoring process into four specialized agents: a Detection Agent that identifies clones using LLM reasoning , a Refactoring Agent that generates Extract Method refactorings guided by retrieval-augmented examples, a Compilation Agent that validates syntax correctness, and a Testing Agent that ensures behavior preservation using auto-generated tests from EvoSuite [19].

Second, we introduce a usefulness checker that explicitly filters out the problematic refactoring patterns identified in a prior empirical study [35]. This checker uses Program Structure Interface (PSI)-based structural analysis and token similarity to detect seven types of unuseful outcomes: incomplete refactoring, excessive refactoring, post-extraction clone deletion, direct clone removal, call-based clone substitution, clone removal by delegation, and fragmentation of logic. Only refactorings that successfully compile, pass all tests, satisfy the usefulness check, and are approved by human developers are applied to the code.

Third, we employ retrieval-augmented generation (RAG) to provide the Refactoring Agent with concrete examples from correct Extract Method refactorings. This approach grounds the LLM's generation process in demonstrated patterns rather than relying solely on its training data.

Importantly, the agents in our workflow do not do each task independently. Instead, we include a feedback loop. If the later component such as Usefulness Checker, Compilation Agent and Testing Agent fail, they will send the error message back to Refactoring Agent, which allows the workflow to refine the results iteratively.

We evaluate our approach on 60 manually constructed clone cases from the Java project JHotDraw. Among these 60 cases, we cover Type-1, Type-2, and Type-3 clones. Here are the definitions for each clone type.

- Type-1: exact copies, except for whitespace, comments, and layouts.
- Type-2: only differ in identifiers, literals or data types.
- Type-3: mostly similar but include added, removed or modified statements.

We first report how effective is multi-agent workflow for code clone detection and refactoring (RQ1). Then we conduct an ablation study to assess the importance of individual components such as retrieval-augmented generation, usefulness checking, and feedback-based iteration (RQ2). Finally, we compare our workflow with baselines (RQ3): a single LLM setting that performs detection and refactoring, and the traditional refactoring tool JDeodorant.

This paper makes the following contributions:

- We designed a multi-agent LLM framework for code clone refactoring.
- We introduce a usefulness checker based on PSI analysis that detects and filters out seven types of unuseful refactoring patterns observed in prior work [35].
- We implement a retrieval-augmented generation that provides detection and refactoring examples to guide the LLM.

The rest of this paper is organized as follows. Section 2 describes our problem statement. Section 3 reviews related work. Section 4 presents our multi-agent framework. Section 5 reports experimental results. Section 6 discusses our findings. Section 7 addresses threats to validity, and Section 8 concludes. Our project with clones, code and results can be found in the replication package [1].

2 PROBLEM STATEMENT

Large Language Models (LLMs) have recently been applied to code clone detection and automated refactoring. While these models can generate refactorings that often compile and pass tests, their behavior is still unreliable. In practice, a single LLM invocation frequently produces incorrect clone detection results or refactorings that preserve behavior but fail to achieve a meaningful Extract Method transformation.

One key difficulty is that clone refactoring involves multiple types of reasoning. The system must first correctly detect duplicated code. It must then apply a structurally sound refactoring. Finally, it must ensure that the refactoring preserves program behavior. A single LLM prompt is rarely sufficient to handle all of these steps in a reliable way.

To address this challenge, we design a multi-agent workflow that separates clone detection, refactoring, compilation, and testing into distinct agents. This workflow allows intermediate validation and feedback, rather than relying on one-shot generation. Based on this design, we study the following research questions.

RQ1: How effective is a multi-agent workflow for clone detection and Extract Method refactoring in terms of correctness and usefulness? Our first research question evaluates the overall effectiveness of the proposed multi-agent workflow. We study whether the workflow can correctly identify code clones and generate refactorings that are both valid and useful. Correctness refers to whether the refactored code successfully compiles and passes

all tests. Usefulness refers to whether the refactoring represents a meaningful Extract Method transformation, rather than incomplete refactoring, excessive extraction, or other degenerate cases.

This research question examines whether decomposing the task into multiple agents leads to more reliable outcomes compared to asking an LLM directly to perform detection and refactoring in a single step.

RQ2: What is the contribution of key validation and guidance components in the multi-agent framework? Our second research question focuses on understanding the role of several core components that support the multi-agent workflow. We design targeted ablation settings that disable specific validation or guidance mechanisms. In particular, we evaluate three variants: (1) removing retrieval-augmented generation (No RAG), (2) removing the usefulness checker (No Usefulness), and (3) disabling the feedback-based iteration mechanism (No Feedback Loop).

These components are not independent agents, but cross-cutting mechanisms that influence how refactorings are generated, filtered, and refined. By selectively disabling them while keeping the overall pipeline intact, we can observe how each mechanism affects end-to-end refactoring success, usefulness, compilation correctness, and test pass rates.

This ablation study allows us to identify which validation and guidance mechanisms are essential for reliability and which mainly provide incremental improvements.

RQ3: How much do usefulness, compilation and test feedback improve refactoring success compared to a single-LLM setting and traditional tool? Our third research question isolates the impact of validation and feedback mechanisms in the workflow. In the multi-agent setting, refactoring proposals are validated through compilation, testing, and an additional usefulness check. When failures occur, error information is fed back to the Refactoring Agent, which can attempt a revised refactoring in the next iteration.

We compare this iterative, feedback-driven setting with two baselines. First, we use a single-LLM baseline, where the model performs clone detection and refactoring in one step without compilation feedback, test execution, or usefulness validation. This baseline represents a one-shot generation setting. Second, we compare against a traditional rule-based refactoring tool, JDeodorant, which applies static analysis and predefined refactoring preconditions to recommend Extract Method transformations. Unlike the LLM-based approaches, JDeodorant does not rely on generative reasoning but instead follows deterministic structural rules.

This comparison allows us to quantify how much structured feedback improves refactoring success, and how the multi-agent workflow performs relative to both a one-shot LLM approach and a traditional static-analysis-based tool.

Scope of Clone Types. This study focuses on Type-1, Type-2, and Type-3 clones. We do not include Type-4 clones in our evaluation. Type-4 clones refer to code fragments that implement the same functionality using substantially different implementations. Detecting and refactoring such clones requires deep semantic understanding of program intent, project architecture, class dependencies, and design constraints.

Our workflow is designed to operate in a paste-triggered setting, in which duplication is introduced via copy–paste operations. In this context, Type-1 to Type-3 clones represent realistic duplication scenarios that developers commonly introduce and subsequently refactor.

Constructing and validating true Type-4 clones would require re-implementing functionally equivalent behavior under different design strategies, which involves extensive project-level reasoning and introduces additional experimental variables. For this reason, we limit the scope of this study to Type-1, Type-2, and Type-3 clones, and leave semantic-level clone refactoring (Type-4) as future work.

Together, these research questions provide a structured evaluation of the proposed multi-agent workflow. They help clarify whether the approach improves clone refactoring reliability, which components matter most, and how feedback-driven iteration compares to one-shot LLM-based refactoring.

3 RELATED WORK

There exist many studies focusing on code clone detection and refactoring. Before LLMs came out, early studies have adopted static analysis techniques to refactor code clones safely without changing the behaviors. For example, Tsantalis et al. [41] proposed a systematic approach to assess clone refactorability by analyzing structural similarity, statement mapping, and refactoring preconditions. Their approach provides strong guarantees on compilation and behavioral preservation. Tools such as JDeodorant build on these principles to recommend Extract Method refactorings based on program structure and code smells.

To better integrate refactoring into developers' daily programming, recent work has explored interactive and just-in-time clone refactoring within the IDE. For example, AntiCopyPaster introduced the idea of monitoring copy and paste actions and detecting code duplication at the moment it is introduced. The original AntiCopyPaster [4] tool recommends Extract Method refactorings based on a classifier that predicts whether a pasted fragment should be extracted. AntiCopyPaster 2.0 [6] extends this approach by introducing white-box detection and customizable rules. They allow developers to tune detection behavior according to their preferences. AntiCopyPaster 3.0 [3] further enhances the system by leveraging IntelliJ PSI for structural clone detection. In this work, they implement a customized Extract Method refactoring pipeline and integrate method name recommendation models. Together, these studies show that just-in-time clone detection can effectively reduce workflow disruption and improve the timeliness of refactoring.

Due to the rise of Large Language Models, recent studies have investigated their abilities for code clone detection as a classification problem. Khajezade et al. [22] conducted an empirical study on applying prompt-based LLMs to detect semantic (Type-4) code clones in both mono-lingual and cross-lingual settings. Their results show that under carefully designed prompts, ChatGPT can achieve performance comparable to or better than fine-tuned deep learning models, particularly in cross-language clone detection scenarios. This work highlights the strong reasoning capability of LLMs for understanding functional similarity between code fragments.

However, this line of work only focuses on clone detection task. There are no further steps after the clones are detected. As a result,

issues such as refactorability, behavior preservation, compilation correctness, and test validation are outside the scope of these approaches. In contrast, our work treats clone detection as the first step in a multi-agent pipeline, where detected clones are passed to the Refactoring Agent to produce Extract Method refactorings.

In more recent studies, LLMs have been applied to refactoring tasks. EM-Assist [33, 34] demonstrates that LLM can generate refactoring suggestions that are similar to developer practices. However, they also discover that certain amount of LLM-generated suggestions are invalid, hallucinated, or not useful without strong filtering. EM-Assist addresses this issue by combining LLM-generated candidates with IDE-based static analysis to eliminate invalid or trivial refactorings before execution. Empirical studies [35] comparing LLMs-based refactoring with traditional tools further show that LLMs perform well in simple clone scenarios, but their performance drops significantly in structurally complex clones. In complex cases, LLMs often produce incomplete refactorings or other patterns which do not meet the requirement of Extract Method refactoring. There are also large amount of refactorings that do not compile or preserve behavior.

Recent work has begun to explore multi-agent architectures for automated refactoring as a way to address the limitations of single-LLM approaches. RefAgent [32] decomposes the refactoring process into specialized agents responsible for planning, execution, testing, and iterative refinement. Their results demonstrate that agent collaboration significantly improves compilation and test success rates compared to single-agent settings. MANTRA [42] improves automated refactoring by integrating contextual retrieval, coordinated multi-agent collaboration, and self-repair mechanisms. Their work also achieves higher rates of test-passing refactorings at the method level.

Multi-agent designs have also been applied to other programming languages and refactoring scenarios. For example, a multi-agent LLM framework for Haskell refactoring [38] introduces role-based agents for structural analysis, code smell detection, strategy planning, execution, testing and debugging of refactoring. They show improvements in complexity, code quality, and performance through staged validation and iterative feedback. Such work demonstrates that decomposing refactoring into explicit analysis, transformation, and validation stages can improve reliability compared to monolithic generation.

In a distinct setting, CoRenameAgent [9] applies a multi-agent framework to coordinated rename refactoring, where agents infer refactoring scope from developer actions and propagate changes across a codebase using trusted IDE refactoring APIs. Together, these studies provide strong evidence that agent specialization, staged validation, and iterative feedback are effective for complex refactoring tasks.

Overall, prior works [12, 13, 22, 43] have studied code refactoring from different angles. Traditional approaches rely on static analysis and predefined rules to assess refactorability and ensure correctness. Recent LLM-based techniques show that models can do code refactoring but their outputs are often unstable. Multi-agent refactoring systems further improve robustness by breaking refactoring into multiple roles, but they usually assume that refactoring targets are already known.

In contrast, our work structures the code clone refactoring tasks. We consider clone refactoring as a step-by-step process which should be carried out by multiple specialized agents. Clone detection serves as the starting point of the workflow. Each following step such as refactoring, compilation, and testing is handled explicitly with clear responsibilities and feedback. In addition, we introduce a usefulness checker that filters out unuseful refactorings introduced in previous work [35] that do not correctly follow Extract Method refactoring. By putting these components into one workflow, our approach brings together ideas from previous studies and treats clone refactoring as a structured step-by-step process rather than a single step.

4 STUDY DESIGN

4.1 Overview

This study adopts a multi-agent workflow to automatically detect and refactor code clones while preserving program behavior. As shown in Figure 1, the system first detects code clones and then generates Extract Method refactorings. Then the generated code is checked by the Usefulness Checker, Compilation Agent and Testing Agent by order.

The system consists of four specialized agents: a Detection Agent, a Refactoring Agent, a Compilation Agent, and a Testing Agent. After the Refactoring Agent produces a candidate transformation, a Usefulness Checker is first applied to evaluate whether the change represents a meaningful Extract Method refactoring. This step filters out unuseful refactorings discovered by previous work [35] prior to further validation.

If the candidate passes the usefulness check, the Compilation Agent compiles the refactored code to ensure that it builds successfully. The Testing Agent then checks whether the refactoring preserves the original program behavior.

The workflow executes sequentially and will iterate when failures occur, with the maximum number of iterations controlled by user settings. When usefulness check, compilation, or testing fails, feedback is generated and provided to the Refactoring Agent. This allows the workflow to gradually improve usefulness and correctness of refactored code across multiple attempts.

We use three related terms to describe how refactorings are evaluated in our workflow: *valid*, *useful*, and *accepted* refactorings.

Valid refactoring. A refactoring is considered valid only if it successfully passes all automated validation steps in the workflow. Validation is performed after a refactoring proposal is generated and it passes the Usefulness Checker.

Validation follows a simple sequence. The refactored code is first compiled using the original project classpath to ensure that it builds correctly. If compilation fails, the refactoring is rejected and the workflow does not proceed to the testing stage.

If compilation succeeds, the workflow continues with testing. The project's test suite is then run on the refactored code. The original project files are not modified during this step. The tests are used to confirm that the refactoring does not change the program's behavior.

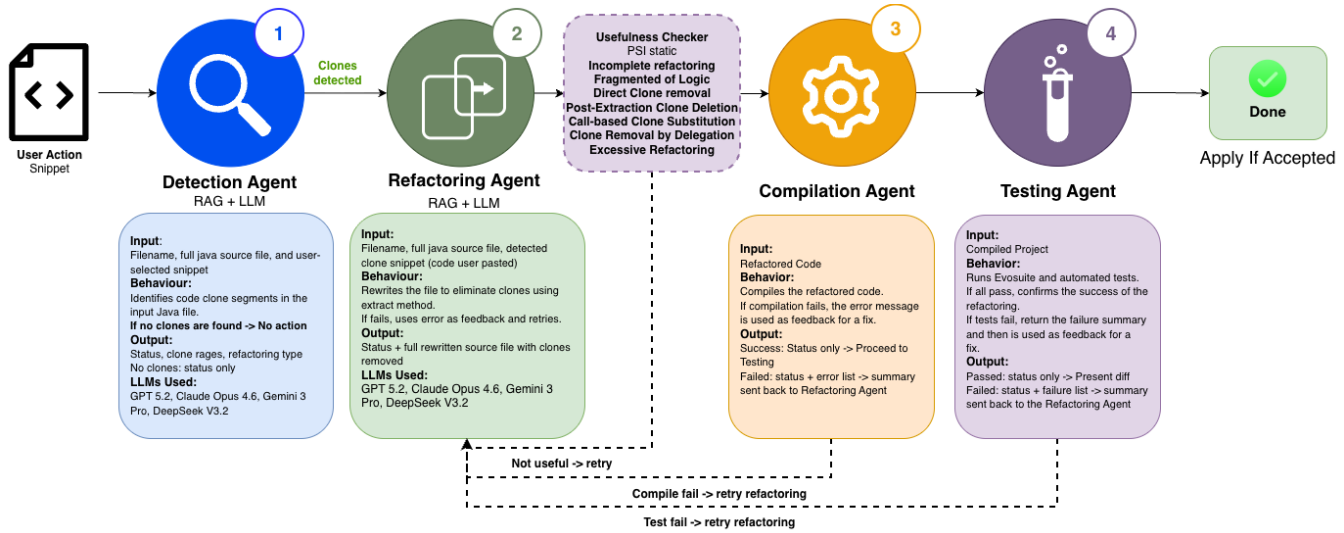

Figure 1: Workflow of Multi-agent Framework.

Only refactorings that pass both Compilation Agent and Testing Agent are considered valid. Refactorings that fail at any validation step are treated as invalid and will not be applied.

Useful refactoring. In this work, a refactoring is considered useful only if it reflects a meaningful and correct Extract Method refactoring.

In Pomian et al.'s works [33, 34], they introduced a mechanism to assess the usefulness of refactorings. We follow this idea and consider usefulness as a structural property of the refactoring. An empirical study by Qian et al. [35] observed that LLM-generated refactorings often compile and pass tests successfully but still fail to produce a proper Extract Method structure. Typical problems include incomplete refactoring, directly deleting clone code or splitting the logic into many small helper methods. Although these changes may remove duplication, they do not match the definition of a proper Extract Method refactoring. Based on these observations, we define a useful refactoring using a clear set of practical rules.

A useful Extract Method refactoring should introduce a single new helper method based on the definition, replace all clone instances with calls to this method, and remove the duplicated logic from the original locations.

The usefulness check is performed right after the Refactoring Agent generates the refactored code, before compilation or testing. This step focuses on checking structural form of the change. The checker compares the code before and after the transformation. It uses token similarity and PSI information to identify improper Extract Method refactoring patterns mentioned in Qian's work [35]. Only refactorings that pass this check continue to the Compilation Agent and Testing Agent.

The Refactoring Agent is also guided by retrieval-augmented generation (RAG). The retrieved examples come from real Extract Method refactorings. If a refactoring fails the usefulness check, it is rejected, and feedback is given to the refactoring agent. This allows the workflow to gradually move toward refactorings that are both correct and meaningful.

Accepted refactoring. A refactoring is considered *accepted* if it passes all stages of the workflow and is finally written back to the original source file. At this point, the refactoring has satisfied the usefulness criteria, successfully compiled, passed all tests and received approval from the human developer.

4.2 Detection Agent

The workflow is triggered when a developer pastes code into the editor. Upon a paste action, the Detection Agent analyzes the updated file that contains the newly pasted code, rather than scanning the entire project or codebase which would consume tokens and potentially exceed the model's token limit. This design allows the tool to operate in an interactive setting and to provide feedback close to the moment when duplication is introduced.

The agent relies on an LLM to reason about code clones, but is constrained to only report clones that are similar to the developer pasted code.

Listing 2: Detection Agent prompt.

```
You are an expert code clone detection assistant.
Analyze the following Java source code from a single file
     named '<FILE_NAME>'.
Your task is to detect meaningful code clones within this
     same file only.
Ignore trivial one-line repetitions and focus on substantial
      code clone.

=== Few-shot examples (from clone database) ===
<OPTIONAL_FEW_SHOT_EXAMPLES>
=== End few-shot examples ===

File source:
'''
<FILE_SOURCE>
```

```
'''

Selected snippet:
'''
<SNIPPET_PASTED>
'''

IMPORTANT: For each detected clone, you MUST include
     cloneCodeA and cloneCodeB as verbatim copies from the
     provided fileSource.
- Do NOT rewrite, reformat, rename variables, or add missing
      context. Copy the exact characters from fileSource.
- If you are not confident you can copy verbatim, set
     cloneCodeA/cloneCodeB to an empty string instead of
     guessing.

Output ONLY a valid JSON object with the following structure
     :
{
  "status": "found_clones" or "no_clones",
  "file": "<FILE_NAME>",
  "clones": [
    {
      "id": "unique_clone_id",
      "ranges": [ { "startLine": int, "endLine": int }, ...
     ],
      "refactorType": "extracted_method" or "extracted_class
     " or other string,
      "reason": "explanation␣of␣why␣this␣clone␣was␣detected"
     ,
      "cloneCodeA": "EXACT␣code␣snippet␣A␣copied␣verbatim␣
     from␣the␣file␣(no␣edits).␣If␣you␣cannot␣copy␣exactly,␣
     return␣an␣empty␣string",
      "cloneCodeB": "EXACT␣code␣snippet␣B␣copied␣verbatim␣
     from␣the␣file␣(no␣edits).␣If␣you␣cannot␣copy␣exactly,␣
     return␣an␣empty␣string"
    },
    ...
  ]
}
Do not include any text outside the JSON object.
```

Listing 2 shows the prompt for Detection agent. The detection prompt contains the following components:

- A task description instructing the model to identify cloned code fragments within a single Java file.
- The full source code of the target file.
- Few-shot examples retrieved from a clone RAG database where examples are from real-world projects.
- Explicit constraints requiring the output clone snippets to be exact substrings of the input code.
- A structured JSON output format specifying clone locations and code snippets.

We include these components to make the detection step more reliable and easier to control. Providing the full source file gives the model the right context, so it does not guess about missing code. The few-shot examples help guide the model toward realistic clone patterns. The substring constraint ensures that the reported clones are copied exactly from the file, which is important for later refactoring and validation. The structured JSON format also makes it easy for the next agent to read and use the results without extra processing.

The output of the Detection Agent is a structured clone description, which serves as the input to the Refactoring Agent.

4.2.1 Algorithm: Detection Agent. Algorithm 1 describes the procedure implemented by the Detection Agent to identify clone candidates within a single Java file. The algorithm reflects the actual workflow logic in our IDE-based implementation.

The Detection Agent is triggered after a paste action. It operates on the updated file that already contains the pasted snippet. The inputs include the project context (if available), the file name f, the full source text S, the pasted snippet S_{snip}, and an LLM caller $\mathcal{L}(\cdot)$.

First, if the pasted snippet is empty, the agent immediately returns a result with status `no_clones` (Lines 3–4). No further analysis is performed.

If the snippet is non-empty, the agent optionally determines the line range (l_s, l_e) of the pasted snippet within the file (Line 6). This step is best-effort and is only used for later filtering. The line range is not required for detection itself.

Next, the agent constructs a detection prompt using the file source, the pasted snippet, and optional retrieved few-shot examples (Line 7). The prompt is then sent to the LLM (Line 8). The raw response is parsed into an internal `DetectionResult` structure (Line 9).

If the parsing fails or the result does not contain any clone candidates, the Detection Agent does not immediately terminate the workflow. Instead, control returns to the outer workflow, which may invoke a lightweight PSI-based fallback detector. The fallback mechanism is implemented at the workflow level and is not part of the Detection Agent itself.

If LLM returns clone candidates successfully and the snippet line range is known, the agent filters the candidates to ensure that at least one reported clone overlaps with the pasted snippet (Lines 16–17). This prevents unrelated clone regions from being reported.

Finally, the agent sets the result status to `found_clones`, attaches the file name, and returns the structured detection result (Lines 19–21). This output is then passed to the Refactoring Agent for further processing.

Algorithm 1 Detection Agent

Input: project context *proj*, file name f, file source S, pasted snippet S_{snip}, LLM caller $\mathcal{L}(\cdot)$
Output: detection result R with `status`, `file`, and `clones`
if S_{snip} is empty **then**
  return {`status` = `no_clones`, `file` = f, `clones` = []}
end if
$(l_s, l_e) \leftarrow$ FINDSNIPPETLINERANGE(S, S_{snip}) ▹ best-effort
$P \leftarrow$ BUILDDETECTIONPROMPT$(proj, S, S_{snip}, f, l_s, l_e)$ ▹ may prepend retrieved few-shot examples
$raw \leftarrow \mathcal{L}(P)$
$R \leftarrow$ PARSEDETECTIONRESULT(raw, f)
if R = `null` **or** R.`clones` is empty **then**
  $R \leftarrow$ PSIFALLBACKDETECT$(proj, S, S_{snip}, f)$
end if
if R = `null` **or** R.`clones` is empty **then**
  return {`status` = `no_clones`, `file` = f, `clones` = []}
end if
if (l_s, l_e) is known **then**
  R.`clones` $\leftarrow$ FILTERBYSNIPPETOVERLAP$(R$.`clones`$, l_s, l_e)$
end if
R.`status` $\leftarrow$ `found_clones`
R.`file` $\leftarrow f$
return R

4.3 Refactoring Agent

The Refactoring Agent generates a refactored version of the source code based on the detected clone. In this study, the agent is strictly limited to applying a single refactoring type: *Extract Method*.

The agent operates on an in-memory copy of the source code. It produces a modified version of the file but does not directly apply changes to disk.

Listing 3: Refactoring Agent prompt with RAG context.

```
You are a Java software engineer.

Your task is to refactor the given Java file to remove
     cloned code.
You must apply Extract Method refactoring only.

Input:
- A Java source file.
- Detected cloned code snippets copied verbatim from the
     file.

Java source file (FULL FILE):
<FILE_SOURCE>

Detected cloned code snippets:
<CLONED_CODE>

Retrieved refactoring examples (for guidance only):
Each example includes a BEFORE code fragment, an AFTER code
     fragment,
and a short note describing the refactoring intent.
Use these examples only as guidance.
Do not copy code directly from them.
Always follow the Extract Method pattern.

[RAG_EXAMPLES]

Instructions:
- Use Extract Method to remove clones.
- Do not apply any other refactoring type.
- Do not rename existing classes, methods, or variables
     unless required by Extract Method.
- Do not modify package or import declarations.
- Restrict all changes to this file only.
- Preserve the original program behavior.
- Keep the refactoring minimal and focused.

Output:
- Return the full updated Java file.
- Output exactly one Java code block.
- Do not include any explanation or text outside the code
     block.

[FEEDBACK]
```

Listing 3 shows the prompt for Refactoring agent. Inspired by previous works [8, 15, 20, 27], the refactoring prompt consists of:

- A role instruction describing the agent as a Java refactoring expert.
- A hard constraint specifying that only Extract Method refactoring is allowed.
- The original source code of the file.
- The detected clone code snippets and their context.
- Retrieved examples of Extract Method refactorings.
- Feedback from previous usefulness, compilation or testing failures, if available.
- An output instruction requiring the full updated Java file as output.

We include each component in the prompt for a certain purpose. The role is assigned as a Java software engineering which could help model be more knowledgeable about Java code. We limit the refactoring technique to only Extract Method, and this prevents models generating unrelated transforms. The full file and cloned code snippets are provided to ensure that the model has a full understanding of the code and dependency. The examples retrieved via RAG can guide the model to generate correct Extract Method refactoring code and reduce the risk of hallucination. Requiring the full updated Java file as output ensures that the result can be directly validated and compiled without additional reconstruction. Finally, feedback from usefulness, compilation or testing failures enables the model to correct the refactoring iteratively.

4.4 Compilation Agent

Following the design idea of RefAgent [32], we incorporate a Compilation Agent which is responsible for validating the syntactic correctness of the generated refactoring code.

Specifically, the agent writes the refactored source code to a temporary directory and invokes the Java compiler using the project's original classpath. Only compilation errors related to the modified file are considered.

The compilation process follows these steps:

- Create a temporary source directory that mirrors the original package structure.
- Write the refactored source code into this directory.
- Invoke the Java compiler with the project classpath.
- Capture and summarize compiler error messages, if any.

If compilation fails, the error summary is sent back to the Refactoring Agent as feedback. If compilation succeeds, the workflow proceeds to the Testing Agent.

4.5 Testing Agent

The Testing Agent ensures that the refactored code preserves the original program behavior. The agent uses EvoSuite [19] to generate unit tests automatically, and then it executes the test files. To ensure that tests exercise the refactored code, the compiled bytecode from the temporary compilation output is placed before the original classes on the classpath. Automatically generated tests do not always fully cover the cloned methods or all relevant execution paths. Among the 60 clone cases in our study, 35 cases have test coverage that directly exercises the cloned methods. For the remaining 25 cases, the generated tests do not directly invoke the cloned fragments. In these situations, we manually inspect the refactored code to verify that the transformation preserves behavior.

The Testing Agent operates using the following information:

- The fully qualified name of the target class.
- A summary of the refactoring that was applied.
- The refactored source code.
- Execution logs from previous test runs, if available.

If all tests pass, we consider the refactoring preserves the original functionality. If tests fail, a summarized test failure report is generated and sent back to the Refactoring Agent. After a number of iterations defined by developers, refactorings that still fail the testing are discarded. This setup guarantees that all tests are executed against the proposed refactoring.

4.5.1 Algorithm: Refactoring with Compilation and Testing Gates. Algorithm 2 presents the workflow that is triggered after the clone is detected. The purpose of this stage is to iteratively generate and validate a correct Extract Method refactoring.

The workflow starts by initializing an empty feedback message (Line 3). This feedback variable saves information from previous failures and is appended to the next refactoring prompt.

The algorithm then enters an iterative loop (Line 4). The maximum number of attempts K is set by the user. In each iteration, the Refactoring Agent constructs a prompt that includes the current source file, the detected clone, retrieved Extract Method examples (via RAG), and feedback from previous usefulness, compilation, or testing failures (Line 5).

The LLM is instructed to produce a full java file with refactored code (Line 6). If the generated output is empty or invalid, the attempt is rejected and the loop continues (Lines 7–10).

Before performing compilation, the generated refactoring is validated by the Usefulness Checker (Line 11). This step analyzes structural information using PSI to ensure that the refactoring effectively eliminates the clones. If the refactoring is classified as not useful, a feedback message is updated, and the algorithm proceeds to the next iteration (Lines 11–14).

If the check passes, then the workflow moves on to the compilation step (Line 15). The refactored source code is written to a temporary directory, which has the same structure as the original package structure. The Java compiler is used with the original project's classpath, and only compilation errors related to the modified file are checked.

In case of compilation failure, the summary of the compiler's log is stored as feedback for the next iteration (Lines 17–20).

Once compilation is successful, the workflow proceeds to the testing phase (Line 21). The bytecode from the temporary directory is moved before the original classes in the classpath. This is to ensure that the tests run on the new refactoring instead of the original code.

The Testing Agent runs automatically generated tests. If the test fails, the feedback summary is stored for the next iteration (Lines 22–24). The testing phase checks for behavioral correctness.

If the refactoring has passed the usefulness test, compilation, and testing, the diff window is shown to the user. The user can choose to confirm whether the patch should be applied to the disk (Lines 26–31).

If the user confirms, the refactored source replaces the original file and the algorithm terminates with success. If the user declines, the refactoring is considered validated but not applied.

If no valid refactoring is found after K attempts, the algorithm terminates and reports failure (Line 33).

4.6 Usefulness Checker

Even if we explicitly instruct the LLM to apply Extract Method only, the generated refactoring is often not a proper Extract Method in practice. In a previous study [35], there were observations that LLMs frequently produce refactorings that look reasonable at the surface level but do not actually remove duplication as the Extract Method refactoring normally does. These outcomes include *incomplete refactoring*, *post-extraction clone deletion*, *direct clone removal*, *call-based clone substitution*, *clone removal by delegation*, and *fragmentation of logic*.

In this work, our goal is to operationalize the observations into concrete and reproducible checks that can be executed automatically. Therefore, we introduce a rule-based usefulness check that runs after refactoring and analyzes the code before and after the change.

The usefulness check uses two complementary signals: (1) normalized token similarity to determine whether clones still exist, and (2) structural evidence extracted from IntelliJ PSI to understand how the refactoring modifies the program structure.

We use two complementary techniques because each captures a different aspect of structural validity under the Extract Method definition used in this study. Normalized token similarity [36] measures whether duplicated logic still exists after refactoring. For example, if two methods remain highly similar after extraction (even after renaming variables or changing literals), it indicates that duplication was not effectively removed. However, token similarity alone cannot explain how the refactoring was performed. Therefore, we also use PSI-based structural analysis to inspect program structure. For instance, PSI allows us to detect whether both clone instances were replaced by calls to the same newly introduced helper method,

Algorithm 2 Refactoring with Usefulness, Compile, and Test Gates (Extract Method only)

Input: file name f, current source S_{cur}, detected clone c, RAG guidance E_{rag}, max attempts K, LLM caller $\mathcal{L}(\cdot)$
Output: success (applied or verified) or failure
feedback ← empty
for *attempt* = 1 **to** K **do**
  fb ← concat(E_{rag}, *feedback*) ▷ prepend RAG, then previous feedback
  S_{cand} ← REFACTORWITHLLM(f, S_{cur}, c, *fb*, $\mathcal{L}$)
  if S_{cand} is empty **then**
    feedback ← "Refactor produced empty or invalid output."
    continue
  end if
  if not ISUSEFULEXTRACTMETHOD(S_{cur}, S_{cand}, c) **then**
    feedback ← "Refactoring is not useful; remove duplication in both places via Extract Method."
    continue
  end if
  log_{comp} ← COMPILEPROPOSEDSOURCETOTEMP(f, S_{cand})
  cr ← ANALYZECOMPILELOG(log_{comp})
  if cr.status ≠ compile_ok **then**
    feedback ← cr.summary
    continue
  end if
  tr ← RUNANDSUMMARIZETESTS(f, S_{cand})
  if tr.status ≠ tests_passed **then**
    feedback ← (tr.summary if exists else tr.raw)
    continue
  end if
  if USERCONFIRMSAPPLYDIFF(S_{cur}, S_{cand}) **then**
    APPLYPATCHTOFILE(f, S_{cand})
    return success
  else
    return success ▷ verified but not applied
  end if
end for
return failure

which is expected in a proper Extract Method refactoring. By combining similarity measurement with structural evidence, we can distinguish between true abstraction and superficial code changes.

Before any structural analysis, we use IntelliJ PSI. PSI is an abstract syntax representation that the IntelliJ platform uses for source code. PSI represents the source code as structured data. An element of PSI represents any concrete construct of the source code, such as a class, method, statement, or expression. PSI represents source code data with resolved information compared to raw text or token-based source code analysis. We use PSI for better analysis of refactorings at the source code level rather than at the surface level.

Step 0: Parse PSI and compute method-level differences. First, we parse the source file in both its before and after versions into IntelliJ PSI trees (lines 1–2 in Listing 4). This allows us to process the source file in a structured form, rather than a raw string.

Next, we gather all method declarations from the two PSI trees (lines 4–5). The methods are stored under unique keys, so we can easily compare the two versions.

Then, we find the set of newly added methods (lines 7–8). We do this by taking all methods from the after version and subtracting the methods that were already present in the before version.

This step is necessary because a proper Extract Method refactoring should introduce a new helper method. If no new method is detected at this stage, it is unlikely that a standard Extract Method has been applied. Listing 4 shows the implementation details.

Listing 4: PSI parsing and added method detection.

```
1 PsiJavaFile beforePsi = parseInMemoryJavaFile(project,
      fileName, beforeSource);
2 PsiJavaFile afterPsi  = parseInMemoryJavaFile(project,
      fileName, afterSource);
3
4 Map<String, PsiMethod> beforeMethods =
      collectAllMethodsByKey(beforePsi);
5 Map<String, PsiMethod> afterMethods  =
      collectAllMethodsByKey(afterPsi);
6
7 Set<String> addedKeys = new HashSet<>(afterMethods.keySet())
      ;
8 addedKeys.removeAll(beforeMethods.keySet());   // AFTER -
      BEFORE
```

Step 1: Measure clone similarity using normalized tokens. To determine whether duplication is removed, we adopt the Jaccard similarity because it quantifies the proportion of shared elements between two sets while ignoring duplicate occurrences. This makes it suitable for detecting structural similarity across normalized token sets [30].

During tokenization, all identifiers are normalized to `ID` (identifier), and all literals are normalized to `LIT` (literal). This normalization removes superficial differences such as variable names and constants, while preserving structural similarity.

Listing 5 shows how similarity is computed for both the before and after versions. In lines 1–3, we first build normalized signatures for the two clone instances in the before version and compute their Jaccard similarity as `simBefore`. In lines 5–7, we repeat the same process for the after version and compute `simAfter`. By comparing `simBefore` and `simAfter`, we can determine whether the duplicated logic still remains after refactoring.

Listing 5: Token similarity with Jaccard.

```
1 Sig aBefore = buildSig(beforeMethods.get(aKey));
2 Sig bBefore = buildSig(beforeMethods.get(bKey));
3 double simBefore = jaccard(aBefore, bBefore);
4
5 Sig aAfter = buildSig(afterMethods.get(aKey));
6 Sig bAfter = buildSig(afterMethods.get(bKey));
7 double simAfter = jaccard(aAfter, bAfter);
```

Step 2: Extract PSI evidence about delegation and method calls. The token similarity alone cannot explain how the refactoring was performed. Therefore, in Step 2 we analyze structural changes using PSI. Listing 6 shows the implementation details with line numbers.

Table 1: Definitions of LLM refactoring hallucinations categories

Category	Definition	General Solution
Incomplete refactoring	The refactoring modifies the code, but most of the duplicated logic still remains in the original methods.	Ensure both clone instances are replaced by calls to the same newly extracted helper method, removing the duplicated logic entirely.
Excessive refactoring	The extracted method includes statements beyond the intended cloned fragment.	Restrict extraction strictly to the duplicated region and avoid introducing unrelated logic into the helper method.
Post-extraction clone deletion	A helper method is introduced, but only one clone is replaced by a call, while the other clone is deleted.	Replace both clone instances with calls to the extracted method instead of deleting one side.
Direct clone removal	One clone instance is deleted without introducing a shared abstraction.	Introduce a common helper method and redirect both clones to it instead of removing code directly.
Call-based clone substitution	One clone is replaced by a call to an existing method instead of extracting a new shared abstraction.	Introduce a dedicated extracted method that captures the shared logic, rather than redirecting to an existing clone.
Fragmentation of logic	The duplicated logic is split into several small methods without consolidating it into one shared abstraction.	Extract the full duplicated fragment into a single coherent helper method to centralize shared behavior.
Clone removal by delegation	The original method is modified to delegate behavior, and the clone calls this modified method instead of a new abstraction.	Introduce a separate extracted method that encapsulates only the duplicated fragment and let both methods delegate to it symmetrically.

In lines 1–2, we call detectDelegate on the two after-version clone methods. This step checks whether a method becomes a thin delegate, meaning its body mainly forwards execution through a method call. A thin delegate is an important signal of Extract Method refactoring, because a proper extraction typically replaces duplicated logic with a call to a shared helper method.

In lines 5-6, we use collectResolvedCalleeKeys to collect the callee sets. Even though a method is not a pure delegate method, we still want to examine which methods are called within its body.

Listing 6: Delegation and callee collection.

```
1 DelegateInfo aDel = detectDelegate(aAfter, afterPsi, cfg);
2 DelegateInfo bDel = detectDelegate(bAfter, afterPsi, cfg);
3
4 // When not pure delegate, still collect callees in the body
    .
5 Set<String> aCalls = collectResolvedCalleeKeys(aAfter,
     afterPsi, cfg);
6 Set<String> bCalls = collectResolvedCalleeKeys(bAfter,
     afterPsi, cfg);
```

Through the combination of the detection of delegation (lines 1-2) and the collection of callees (lines 5-6), it is possible to infer whether the duplicated code has been replaced by a call to a common method, redirected to an existing method, or has been structurally altered in some other fashion. These PSI-based features enable us to differentiate between actual Extract Method refactoring and other superficial structural changes.

Using the structural information gathered in Step 2, the common failure modes identified in previous work [35] can be described.

Listing 7: Incomplete refactoring example

```
/* Code fragment 1 in LineAndShapeRenderer */
double previous = previousValue.doubleValue();
double x0;
if (this.useSeriesOffset) {
    x0 = domainAxis.getCategorySeriesMiddle(
        dataset.getColumnKey(column - 1),
        dataset.getRowKey(row), dataset,
        this.itemMargin, dataArea,
        plot.getDomainAxisEdge());
}
else {
    x0 = domainAxis.getCategoryMiddle(column - 1,
        getColumnCount(), dataArea,
        plot.getDomainAxisEdge());
}
double y0 = rangeAxis.valueToJava2D(previous, dataArea,
    plot.getRangeAxisEdge());
Line2D line = null;
if (orientation == PlotOrientation.HORIZONTAL) {
    line = new Line2D.Double(y0, x0, y1, x1);
}
else if (orientation == PlotOrientation.VERTICAL) {
    line = new Line2D.Double(x0, y0, x1, y1);
}
g2.setPaint(getItemPaint(row, column));
g2.setStroke(getItemStroke(row, column));
g2.draw(line);

/* Code fragment 2 in StatisticalLineAndShapeRenderer */
double previous = previousValue.doubleValue();
double x0;
if (getUseSeriesOffset()) {
    x0 = domainAxis.getCategorySeriesMiddle(
        dataset.getColumnKey(column - 1),
        dataset.getRowKey(row), dataset,
        getItemMargin(), dataArea,
        plot.getDomainAxisEdge());
}
else {
    x0 = domainAxis.getCategoryMiddle(column - 1,
        getColumnCount(), dataArea,
        plot.getDomainAxisEdge());
}
double y0 = rangeAxis.valueToJava2D(previous, dataArea,
    plot.getRangeAxisEdge());
Line2D line = null;
if (orientation == PlotOrientation.HORIZONTAL) {
    line = new Line2D.Double(y0, x0, y1, x1);
}
else if (orientation == PlotOrientation.VERTICAL) {
    line = new Line2D.Double(x0, y0, x1, y1);
}
g2.setPaint(getItemPaint(row, column));
g2.setStroke(getItemStroke(row, column));
g2.draw(line);

/* Extracted method */
protected void drawLineBetweenPoints(Graphics2D g2,
        PlotOrientation orientation,
        double x0, double y0,
        double x1, double y1,
        int row, int column) {

    Line2D line = null;

    if (orientation == PlotOrientation.HORIZONTAL) {
        line = new Line2D.Double(y0, x0, y1, x1);
    }
```

```
    else if (orientation == PlotOrientation.VERTICAL) {
        line = new Line2D.Double(x0, y0, x1, y1);
    }

    g2.setPaint(getItemPaint(row, column));
    g2.setStroke(getItemStroke(row, column));
    g2.draw(line);
}
```

Incomplete refactoring. As can be seen in Table 1, in an incomplete refactoring failure mode, an attempt is made to refactor a code, but the duplicated code is not effectively removed [35]. Listing 7 shows an example from previous empirical study [35]. The extract method drawLineBetweenPoints does not fully extract the whole cloned code. In these instances, the output of the refactoring is structurally different but is still very similar to both instances of code duplication. Therefore, it is clear that the refactoring process did not actually accomplish its primary objective: removing code duplication using a common abstraction.

In terms of practicality, an incomplete refactoring is a failure mode because it does not actually offer a structural advantage. For instance, a model could add a new helper method, make method calls, but still leave both clone bodies almost entirely intact.

To detect incomplete refactoring, we compare the similarity of the clone instances before and after refactoring using normalized token-based Jaccard similarity. If the similarity in the after version remains high, this means the clone is still present.

In our implementation, we use a similarity threshold of 0.8 for the after-version normalized token Jaccard similarity (i.e., `simAfter` ≥ 0.8). We choose 0.8 because it means that at least 80% of the normalized tokens are shared between the two clone instances, which suggests that the duplicated logic remains largely unchanged. A lower threshold would make the check more sensitive, and it may label partially improved refactorings as incomplete. In contrast, a much higher threshold (e.g., 0.9 or above) would mostly detect near-identical fragments and could miss cases where duplication is still substantial. In general, 0.8 is a practical balance between sensitivity and strictness to identify incomplete refactorings.

However, high similarity alone is not sufficient, since no refactoring may have been attempted at all. Therefore, we additionally require evidence of a refactoring attempt derived from PSI analysis.

Specifically, we treat a refactoring attempt as having occurred if at least one of the following conditions holds: (1) a new method is introduced in the after version, (2) at least one clone instance becomes a method call, or (3) the clone instances share one or more method calls that did not exist before. These signals indicate that the model tried to restructure the code, even if the result was ineffective.

Listing 8 shows the detection logic. In line 1, the algorithm first checks whether `simAfter` is greater than or equal to the predefined threshold cfg.cloneSimilarityAfterStill. This condition indicates that the similarity between the two clone instances remains high after refactoring, suggesting that duplication may still exist.

In lines 2-5, the variable `refactorAttempt` is calculated. This boolean expression is a collection of PSI-based structural evidence. Line 3 checks if new methods were added (addedKeys). Line 4 checks if either of the clone instances is a delegate method. Line 5 checks if the two clone instances share new method calls (sharedCalls). The overall evidence points to a model refactoring attempt.

In lines 7-9, the two conditions are combined. If the similarity is still high (line 1) and there is structural evidence for a refactoring attempt (lines 2-5), then the refactoring is classified as INCOMPLETE_REFACTORING. The decision is returned in line 8, marked as not useful in a `PairOutcome`.

This design is conservative in that we are not penalizing cases where no refactoring attempt is made, and we are still effectively filtering out cases where there is a structural refactoring attempt that failed to meaningfully eliminate duplication.

Listing 8: Incomplete refactoring detection.

```
1  if (simAfter >= cfg.cloneSimilarityAfterStill) {
2      boolean refactorAttempt =
3          (!addedKeys.isEmpty()) ||
4          aDel.isDelegate || bDel.isDelegate ||
5          (!sharedCalls.isEmpty());
6
7      if (refactorAttempt) {
8          return new PairOutcome(Strategy.
       INCOMPLETE_REFACTORING, false);
9      }
10 }
```

Listing 9: Excessive refactoring example

```
/* Code fragment 1 in Hour */
if (o1 instanceof Hour) {
    Hour h = (Hour) o1;
    result = getDay().compareTo(h.getDay());
    if (result == 0) {
        result = this.hour - h.getHour();
    }
}
else if (o1 instanceof RegularTimePeriod) {
    result = 0;
}
else {
    result = 1;
}

/* Code fragment 2 in Minute */
if (o1 instanceof Minute) {
    Minute m = (Minute) o1;
    result = getHour().compareTo(m.getHour());
    if (result == 0) {
        result = this.minute - m.getMinute();
    }
}
else if (o1 instanceof RegularTimePeriod) {
    result = 0;
}
else {
    result = 1;
}

/* Extracted method */
protected int compareTimePeriod(Object o1, RegularTimePeriod
      period,
```

```
                                int thisValue, int
     otherValue) {
    int result;
    if (o1.getClass().equals(period.getClass())) {
        RegularTimePeriod other = (RegularTimePeriod) o1;
        result = thisValue - otherValue;
    }
    else if (o1 instanceof RegularTimePeriod) {
        result = 0;
    }
    else {
        result = 1;
    }
    return result;
}
```

Excessive refactoring. As shown in Table 1, we can see the definition of Excessive refactoring. Excessive refactoring refers to a situation where the extracted helper method is larger than the original cloned fragments. Listing 9 also shows an example of excessive refactoring from Qian et al.'s work [35]. The extracted method compareTimePeriod has more lines of code than just two cloned code fragments. This kind of refactoring may compile and run successfully, but it is not a proper situation for an Extract Method refactoring.

To identify this kind of refactoring, we can start by obtaining the size of the original clone fragments (Lines 1-3, Listing 10). We can get this by counting the number of non-empty code lines of each clone fragment. Then we can get the larger one as a baseline.

Then we can get the extracted helper method by PSI (Lines 6-15). We can get this by intersecting the callees of two clone instances (Lines 6-7) and checking if any of these callees is a newly added method (Lines 10-14).

Finally, we compare the size of the extracted method body with the baseline (Lines 17–22). We allow a small tolerance of two lines (Line 21) to ignore trivial formatting differences. If the extracted method contains more lines than the baseline plus tolerance (Line 22), we classify the refactoring as *Excessive Refactoring* and mark it as not useful (Lines 23–30).

Listing 10 shows the complete detection logic.

Listing 10: Excessive refactoring detection.

```
1  int baseLinesA = countNonEmptyCodeLines(beforeA);
2  int baseLinesB = countNonEmptyCodeLines(beforeB);
3  int baseLines = Math.max(baseLinesA, baseLinesB);
4
5  // Identify extracted method: shared callee that is newly
       added
6  Set<String> shared = new HashSet<>(evA.calleeKeys);
7  shared.retainAll(evB.calleeKeys);
8
9  String extractedKey = null;
10 for (String k : shared) {
11     if (addedMethodKeys.contains(k)) {
12         extractedKey = k;
13         break;
14     }
15 }
16
17 if (extractedKey != null) {
18     PsiMethod extracted = afterMethods.get(extractedKey);
19     int extractedLines = countMethodBodyNonEmptyLines(
       extracted);
20
21     int tolerance = 2; // ignore trivial formatting
       differences
22     if (extractedLines > baseLines + tolerance) {
23         return new UsefulnessResult(
24             false,
25             35,
26             Strategy.EXCESSIVE_REFACTORING,
27             List.of(Reason.EXCESSIVE_REFACTORING_DETECTED),
28             "extractedLines=" + extractedLines +
29             ", baseFragmentLines=" + baseLines
30         );
31     }
32 }
```

Listing 11: Post-extraction clone deletion example

```
static public DrawingView
     findDrawingViewInChildren(Container container) {
    Component[] components = container.getComponents();
    for (Component component : components) {
        if (component instanceof DrawingView) {
            return (DrawingView) component;
        } else if (component instanceof Container) {
            DrawingView foundView =
     findDrawingViewInChildren((Container) component);
            if (foundView != null) {
                return foundView;
            }
        }
    }
    return null;
}

- static public DrawingViewgetDrawingView_Cloned
-(Container container) {
- DrawingView o = null;
- Component[] c = container.getComponents();
- for (int ij = 0; ij < c.length; ij++) {
- if (c[ij] instanceof DrawingView) {
- return (DrawingView)c[ij];
- }
- else if (c[ij] instanceof Container) {
- o = getDrawingView((Container)c[ij]);
- if (o != null) {
- return o;
- }
- }
- }
- return null;
```

```
- }

static public DrawingView getDrawingView(Container
     container) {
    return findDrawingViewInChildren(container);
}
```

Post-extraction clone deletion. Listing 11 shows an example of post-extraction clone deletion from previous empirical study [35]. It refers to a refactoring pattern where a helper method is introduced, but the duplicated code is not refactored symmetrically as shown in Table 1. Instead of replacing both clone instances with calls to the extracted method, only one clone instance is replaced, while the other clone instance is deleted. As a result, the duplicated logic is not uniformly abstracted into a single method.

Although a new helper method exists, the refactoring does not preserve the structures expected from Extract Method, where all duplicated instances should be replaced by calls to the same extracted method. Therefore, even though a new method is added, this refactoring is not considered useful under its definition.

To detect post-extraction clone deletion, we combine three signals from the after version, as shown in Listing 12. First, in Lines 1–2, we check whether each clone instance still exists by testing whether `aAfter` or `bAfter` is `null`. In Line 4, we use an exclusive-or condition to ensure that exactly one clone instance is deleted. This prevents cases where both sides are removed. Next, in Lines 5–6, we retrieve the remaining clone instance from `afterMethods`. In Lines 8–9, we invoke `detectDelegate` to determine whether the remaining method has become a delegate, meaning its body mainly consists of a method call. Finally, in Line 11, we check two additional conditions: whether new methods were introduced (`!addedKeys.isEmpty()`) and whether the remaining method is confirmed as a delegate. Only when all these conditions hold do we classify the refactoring as POST_EXTRACTION_DELETION in Line 12 and mark it as not useful.

Listing 12: Post-extraction clone deletion detection.

```
1  boolean aDeleted = (aAfter == null);
2  boolean bDeleted = (bAfter == null);
3
4  if (aDeleted ^ bDeleted) { // exactly one side deleted
5      String remainingKey = aDeleted ? bKey : aKey;
6      PsiMethod remaining = afterMethods.get(remainingKey);
7
8      DelegateInfo del = (remaining == null) ? null
9          : detectDelegate(remaining, afterPsi, cfg);
10
11     if (!addedKeys.isEmpty() && del != null && del.
        isDelegate) {
12         return new PairOutcome(Strategy.
        POST_EXTRACTION_DELETION, true);
13     }
14 }
```

Listing 13: Direct clone removal example

```
1  static public int direction(int x1, int y1, int x2, int y2)
       {
2      int direction = 0;
3      int vx = x2 - x1;
4      int vy = y2 - y1;
5      if (vy < vx && vx > -vy) {
6          direction = EAST;
7      }
8      else if (vy > vx && vy > -vx) {
9          direction = NORTH;
10     }
11     else if (vx < vy && vx < -vy) {
12         direction = WEST;
13     }
14     else {
15         direction = SOUTH;
16     }
17     return direction;
18 }
19 - static public int direction_Cloned(int x1, int y1,
20 - int x2, int y2) {
21 - int d = 0;
22 - int x = x2 - x1;
23 - int y = y2 - y1;
24 - if (y < x && x > -y) {
25 - d = EAST;
26 - }
27 - else if (y > x && y > -x) {
28 - d = NORTH;
29 - }
30 - else if (x < y && x < -y) {
31 - d = WEST;
32 - }
33 - else {
34 - d = SOUTH;
35 - }
36 - return d;
37 - }
```

Direct clone removal. In Table 1, direct clone removal refers to cases where duplicated code is eliminated by simply deleting one or both clone instances, without introducing any shared abstraction. Example from prior work [35] is shown in Listing 13. In other words, the refactoring removes code rather than restructuring it. This pattern was also reported as a common outcome that reduces duplication but does not follow the Extract Method pattern.

From a refactoring perspective, direct clone removal is different from Extract Method because no helper method is created to encapsulate the shared logic. As a result, there is no reusable abstraction that captures the intent of the duplicated code. Therefore,

although duplication may be reduced, such changes are not considered useful in our workflow based on the definition of Extract Method refactoring.

To detect direct clone removal, we compare the presence of clone instances between the before- and after versions. If a clone instance cannot be located in the after version, we treat it as deleted. This can happen in two ways: either one side is removed, or both sides are completely removed.

However, deleting one side after introducing a helper method corresponds to a different pattern, namely post-extraction clone deletion. Therefore, we first check and exclude the post-extraction case. If the removal cannot be explained by post-extraction deletion, we classify it as direct clone removal.

Listing 14 shows the detection logic in detail.

In Line 1, the code checks whether exactly one clone instance is deleted using the exclusive-or condition. This means that one side is missing while the other still exists. If this condition holds, and the case was not already classified as post-extraction clone deletion, the refactoring is immediately labeled as `DIRECT_REMOVAL` in Line 3.

In Line 5, the second branch checks whether both clone instances are deleted using the condition. If both sides are removed, the refactoring is again classified as `DIRECT_REMOVAL` in Line 6. In both branches (Lines 3 and 6), the returned result marks the refactoring as not useful, because no shared helper method is introduced and no Extract Method structure is formed.

Listing 14: Direct clone removal detection.

```
1 if (aDeleted ^ bDeleted) {
2     // if not classified as POST_EXTRACTION_DELETION above
3     return new PairOutcome(Strategy.DIRECT_REMOVAL, true);
4 }
5 if (aDeleted && bDeleted) {
6     return new PairOutcome(Strategy.DIRECT_REMOVAL, true);
7 }
```

Call-based clone substitution. As listing in Table 1, call-based clone substitution refers to cases where both clone instances are replaced by calls to the same existing method, rather than to a newly extracted helper method. Listing 1 also shows an example of this pattern from previous work [35]. In this situation, the duplicated logic is removed from the two clone locations, but no new abstraction is introduced by the refactoring. Instead, the refactoring simply redirects both clone sites to an already-existing implementation.

Although duplication is reduced, it does not correspond to an Extract Method refactoring, because the refactoring does not create a new method that captures the shared logic. Therefore, we treat call-based clone substitution as not useful for our workflow.

To detect this case, we rely on PSI-based delegation and callee analysis. First, we check whether both clone sides become delegate methods after refactoring. A delegate method is a method whose body mainly consists of method calls rather than original logic. Second, we collect the set of resolved callee methods for each clone side and compute their intersection. If both sides delegate to at least one common callee, this indicates that the duplicated logic has been substituted by a method call.

However, to distinguish this case from a proper Extract Method, we further check whether the shared callee corresponds to a newly added method. If all shared callees already existed in the before version of the file, then the refactoring did not introduce a new abstraction. In this case, we classify the refactoring as call-based clone substitution.

Listing 15 shows the implementation in detail. In Line 1, the code first checks whether both clone sides are delegate methods using the condition. This ensures that both methods mainly consist of calls rather than original logic. In Lines 2–3, it computes the intersection of the callee sets from both sides. The variable `shared` contains the methods that are invoked by both clone instances. In Line 5, the condition `!shared.isEmpty()` verifies whether the two sides share at least one common callee. In Line 6, the code checks whether any of the shared callees correspond to newly added methods using `anyMatch(addedKeys::contains)`.Finally, in Lines 8–9, if none of the shared callees are newly introduced (i.e., `!sharedHasNew`), the refactoring is classified as `DELEGATION_TO_EXISTING`. In this case, the duplicated logic is redirected to an existing method rather than extracted into a new abstraction, and the refactoring is therefore marked as not useful.

Listing 15: Call-based clone substitution detection.

```
1  if (aDel.isDelegate && bDel.isDelegate) {
2      Set<String> shared = new HashSet<>(aDel.calleeKeys);
3      shared.retainAll(bDel.calleeKeys);
4
5      if (!shared.isEmpty()) {
6          boolean sharedHasNew = shared.stream().anyMatch(
       addedKeys::contains);
7
8          if (!sharedHasNew) {
9              return new PairOutcome(Strategy.
       DELEGATION_TO_EXISTING, true);
10         }
11     }
12 }
```

Listing 16: Clone removal by delegation example

```
public void rotate(double angle) {
    willChange();
    fRotation = nearestRotationIndex(angle);
    changed();
}

public void rotate_Cloned(double angle) {
  rotate(angle);
}

private int nearestRotationIndex(double angle) {
    double dist = Double.MAX_VALUE;
    int best = 0;
    for (int i = 0; i < rotations.length; ++i) {
        double d = Math.abs(angle - rotations[i]);
        if (d < dist) {
            dist = d;
            best = i;
        }
    }
    return best;
}
```

Clone removal by delegation. Clone removal by delegation describes cases where delegation is applied in an asymmetric or inconsistent way in clone instances as shown in Table 1. Listing 16 demonstrates an example from Qian et al.'s work [35]. Instead of both clone locations being replaced by calls to the same extracted method, delegation happens only on one side, or the two sides delegate to different helper methods. As a result, duplication is not unified into a single abstraction, and the refactoring does not correspond to a proper Extract Method.

This pattern is part of the non-useful refactoring outcomes, and we explicitly detect it to avoid accepting such changes. The key observation is that delegation alone is not sufficient: both clone sides must delegate to the same callee to truly remove duplication.

Our implementation relies on PSI-based call resolution. For each clone side in the after version, we collect the set of resolved callees appearing in the method body. We then compare the two callee sets to check whether there exists a shared callee that both sides invoke.

Listing 17 presents the detection logic with explicit line numbers. In Lines 1–2, we compute the intersection of the callee sets from both clone sides. The set `sharedCalls` contains the methods that are invoked by both instances in the after version. In Line 4, the boolean variable `anyDelegate` checks whether at least one side is classified as a delegate method (i.e., its body mainly consists of method calls). In Line 5, `noSharedCallee` verifies whether the two sides do not share any common callee. Finally, in Lines 7–8, if at least one side delegates (`anyDelegate`) and there is no shared callee between them (`noSharedCallee`), the refactoring is classified as clone removal by delegation and marked as not useful.

This condition captures asymmetric or inconsistent delegation, where delegation is applied but does not unify duplicated logic into a single shared abstraction.

Listing 17: Clone removal by delegation.

```
Set<String> sharedCalls = new HashSet<>(aCalls);
sharedCalls.retainAll(bCalls);

boolean anyDelegate = aDel.isDelegate || bDel.isDelegate;
boolean noSharedCallee = sharedCalls.isEmpty();

if (anyDelegate && noSharedCallee) {
    return new PairOutcome(Strategy.UNKNOWN, false);
}
```

Listing 18: Fragmentation of logic example

```
void buildChildMenus() {
    JInternalFrame[] array = desktop.getAllFrames();
    updateWindowCommands(array.length > 0);

    if (array.length == 0) {
        return;
    }

    addSeparator();
    createChildMenuItems(array);
}

+ private void updateWindowCommands(boolean enabled) {
+ cascadeCommand.setEnabled(enabled);
+ tileHCommand.setEnabled(enabled);
+ tileVCommand.setEnabled(enabled);
+ arrangeHCommand.setEnabled(enabled);
+ arrangeVCommand.setEnabled(enabled);
+ }
+ private void createChildMenuItems
+ (JInternalFrame[] frames) {
+ for (int i = 0; i < frames.length; i++) {
+ ChildMenuItem menu = new ChildMenuItem(frames[i]);
+ menu.setState(i == 0);
+ menu.addActionListener(createChildMenuItemListener());
+ menu.setIcon(frames[i].getFrameIcon());
+ add(menu);
+ }
+ }
```

Fragmentation of logic. Table 1 shows the definition of fragmentation of logic. It refers to cases where a duplicated computation is no longer centralized into a single extracted method, but instead scattered into multiple helper calls. Listing 18 also shows an example from the empirical study [35]. Rather than reducing duplication through one clear Extract Method refactoring, the logic becomes split across several smaller methods, making the refactoring harder to understand and maintain. This refactoring outcome was explicitly reported as an undesired pattern in prior work [35], and motivates us to detect and filter it.

In our workflow, we detect fragmentation by analyzing method call structure in the after version. Specifically, we examine which helper methods are called by the two clone locations after refactoring. If both sides invoke multiple shared callees, this indicates that the original duplicated logic has been decomposed into several pieces rather than extracted as a single cohesive unit.

Listing 19 presents the detection logic. In Lines 1–2, we compute the intersection of the resolved callee sets for the two clone sides (`aCalls` and `bCalls`). The resulting set `sharedCalls` represents helper methods invoked by both sides after refactoring. In Lines 4–7, if the number of shared callees is at least two and the token-level similarity has been reduced, we classify the refactoring as fragmentation. In Lines 9–16, we additionally handle the case where both clone sides become delegate methods. If both are delegates and still share two or more callees (Lines 11–14), the transformation is again classified as fragmentation.

All cases detected through Lines 4–7 or Lines 11–14 are marked as not useful, since they do not correspond to a standard Extract Method refactoring that introduces a single shared abstraction.

Listing 19: Fragmentation of logic detection.

```java
Set<String> sharedCalls = new HashSet<>(aCalls);
sharedCalls.retainAll(bCalls);

if (sharedCalls.size() >= 2 &&
    simAfter <= cfg.cloneSimilarityAfterReduced) {
    return new PairOutcome(Strategy.FRAGMENTATION, true);
}

if (aDel.isDelegate && bDel.isDelegate) {
    Set<String> shared = new HashSet<>(aDel.calleeKeys);
    shared.retainAll(bDel.calleeKeys);
    if (shared.size() >= 2) {
        return new PairOutcome(Strategy.FRAGMENTATION, true);
    }
}
```

Final decision. After classification, we apply a strict decision rule. Only refactorings that correspond to a standard Extract Method and do not fall into any of the above categories are considered useful. All other outcomes are rejected, even if the refactored code compiles and passes tests.

4.7 Retrieval-Augmented Generation

The workflow uses retrieval-augmented generation to support both clone detection and refactoring. The purpose of this component is to provide concrete examples from prior cases, rather than to define refactoring rules or make decisions. Retrieved examples are used as contextual guidance and are injected into prompts when available.

To support retrieval-augmented detection, we construct a clone example database used as few-shot context for the Detection Agent. The database is designed to expose the model to realistic clone patterns observed in prior tools and benchmarks, rather than to define explicit detection rules. For Type-1, Type-2, and Type-3 clones, we collect all the examples from JDeodorant [41] in nine open-source Java projects: Apache Columba, EMF, Hibernate, jEdit, JMeter, JRuby, SQuirreL SQL Client, and JFreeChart. These projects are commonly used in clone research and contain developer-confirmed refactoring cases. We further include Type-1 to Type-3 clone pairs from BigCloneBench [39] and GPT-Benchmark [2] to increase diversity and difficulty. To cover semantic clones (Type-4), we extend the database with examples from LLM-for-CCD [22] and BigCloneBench [39]. In GPT-Benchmark, clone types are classified using Claude Opus 4.5, since it was the newest model available at the time and showed better performance than GPT-5.1 on clone type classification. By adding those examples, the Detection Agent is exposed to a broad range of clone patterns while maintaining a consistent prompt structure.

We also construct a separate example database to support retrieval-augmented refactoring. This database is used only by the Refactoring Agent and contains concrete Extract Method refactoring examples. For Type-1, Type-2, and Type-3 clones, we randomly sample 50 clone instances for each type from the JDeodorant projects. Each selected clone instance includes the original duplicated code and its corresponding refactored version, where duplication is removed by extracting a shared helper method. For Type-4 (semantic) clones, we select 50 clone pairs from LLM-for-CCD. Because LLM-for-CCD does not provide refactoring ground truth, we apply Extract Method refactoring using GPT-5.1, which is different from the model used in our experiments because it was the newest model available when we created the RAG database. The generated refactorings are then manually inspected to verify that the extracted method is behavior-preserving and that duplication is correctly removed. Only refactorings that follow a standard Extract Method pattern and introduce a single shared abstraction are included. This process ensures that the refactoring database contains consistent, non-trivial examples across clone types.

Two separate example collections are used. For clone detection, the example collection consists of labeled pairs of code fragments. These pairs cover a wide range of cases, including Type-1, Type-2, Type-3, and Type-4 clones, as well as non-clone examples. Including both positive and negative cases allows the Detection Agent to observe different forms of similarity and to better distinguish true clones from coincidental resemblance.

For refactoring, a different example collection is used. Each entry contains a pair of cloned code fragments together with a refactored version produced using Extract Method. All examples in this collection follow the same refactoring strategy, which is consistent with the constraint enforced in the Refactoring Agent. These examples represent concrete refactoring results observed in practice.

The way retrieved examples are selected differs between detection and refactoring. In the detection stage, a small and fixed number of examples is included as few-shot demonstrations. The examples are chosen to cover different clone types as well as non-clone cases, and are inserted near the beginning of the detection prompt. This design keeps the prompt concise while still exposing the model to diverse detection scenarios.

In the refactoring stage, inspired by Mantra [42], the selection of examples is performed in two steps to strike a balance between efficiency and structural relevance.

In the first step, a simple lexical similarity test is performed by comparing the number of common tokens in the query clone and the example code. The tokens are identified by normalizing the identifiers and literals in the code.

In the second step, a semantic similarity test is performed by converting the code into vector embeddings to calculate the cosine similarity between the query clone and the example code.

The results of both the tests are fused together using the reciprocal rank fusion (RRF) technique [14] to perform the final selection of the examples to be included in the prompt. Instead of using the similarity scores of the examples, the RRF technique relies on the rank of the examples in the final selection of k examples to be included in the prompt.

In the refactoring prompt, each selected example is presented in a consistent before-and-after format. The examples are clearly separated from the target source code and are explicitly marked as guidance only. The Refactoring Agent is instructed not to copy code directly from the examples, but to follow the demonstrated refactoring pattern.

Across both stages, the use of retrieved examples is optional. If no suitable examples are found, the workflow continues without retrieval-based generation. Correctness is not enforced by the retrieved examples, but by grounding checks in detection and by compilation and testing in refactoring.

4.8 Qualitative Analysis

All experiments in RQ1, RQ2, and RQ3 were performed manually by two authors. For each clone case and each model configuration, the authors independently ran the models, applied the generated refactoring to the JHotDraw project, and recorded the outcomes, including detection, usefulness, compilation, testing, and final success.

The clone cases used in our study were manually constructed. We selected real methods from different packages of the JHotDraw project and then created cloned versions by copying and modifying the original code. We selected JHotDraw as the subject system because it is a well-known open-source Java project and has been widely used in code clone researches [10, 24, 25, 31, 40]. We first inspected the source code and selected real methods from different packages and classes. Then, we manually created cloned methods by copying and modifying the original code. This design allowed us to control the type of clone, the location of the code, and the difficulty level. It also ensured that the ground truth of each clone pair was clear, which is important for evaluating detection and refactoring behavior.

We constructed three types of clones. Type-1 clones are exact copies of the original method. Type-2 clones contain renamed variables, changed data types, or modified literal values, while keeping the overall structure the same. Type-3 clones include small insertions, deletions, or internal modifications inside the method body. By including these different clone types, we aimed to observe how structural differences influence the behavior of the models and the stability of the refactoring process.

We evaluated four language models: GPT-5.2, Gemini 3 Pro, Claude Opus 4.6, and Deepseek v3.2. These models were selected because they represent different model families and capability levels. By comparing these models under the same workflow, we are able to observe how model characteristics interact with structural constraints such as the usefulness checker and the feedback loop.

The usefulness evaluation followed the structural criteria defined in Section 4.6. Each refactoring was examined to determine whether it satisfied the Extract Method definition adopted in this study.

In cases where the two authors disagreed on the usefulness judgment, the case was discussed until consensus was reached.

Listing 20: Example of Refactoring With Disagreement.

```java
public void dragOver(DropTargetDragEvent dtde) {
    //log("DropTargetDragEvent-dragOver");
    updateLastMousePositionForDragOver(dtde);
}

public void dragOver_Cloned(DropTargetDragEvent dtde) {
    //log("DropTargetDragEvent-dragOver");
    updateLastMousePositionForDragOver(dtde);
}

private void updateLastMousePositionForDragOver(
     DropTargetDragEvent dtde) {
    if (supportDropTargetDragEvent(dtde) == true) {
        int x = dtde.getLocation().x;
        int y = dtde.getLocation().y;
        if ((Math.abs(x - fLastX) > 0) ||
            (Math.abs(y - fLastY) > 0)) {

            //FigureEnumeration fe = view().
    selectionElements();
            //while (fe.hasNextFigure()) {
            //    fe.nextFigure().moveBy(x - fLastX, y -
    fLastY);
            //    System.out.println("moving Figures " +
    view());
            //}
            //view().drawing().update();

            fLastX = x;
            fLastY = y;
        }
    }
}
```

One representative disagreement case is shown in Listing 20. For this refactoring, the two authors initially reached different judgments. One author considered it a correct Extract Method refactoring, since the duplicated logic was fully extracted into a shared helper method. The other author classified it as incomplete refactoring, arguing that the original clone methods still contained residual content.

After discussion, it was observed that the remaining lines were only commented code and did not contain any executable statements that could affect program behavior. Since no functional logic was left in the original methods, the extraction did not violate the structural definition adopted in this study. The two authors therefore reached consensus and labeled this case as a valid and useful Extract Method refactoring.

In addition to manual evaluation, we conducted statistical analysis to examine whether the observed performance differences among models are statistically significant. Since each clone case is evaluated by all models under the same experimental setting, we first applied Cochran's Q test [11] to assess overall differences across multiple related samples. When the overall test indicated significant differences, we performed pairwise McNemar tests [28] for comparisons between model pairs. Bonferroni correction [17] was applied to control the family-wise error rate for multiple comparisons. We used a significance level of 0.05 before correction. For statistically significant pairwise results, we further computed effect sizes to quantify the magnitude of differences. These statistical analyses complement the qualitative inspection and help ensure that improvements observed in the multi-agent workflow are not due to random variation.

5 EXPERIMENTAL RESULTS

In this section, we discuss the results and findings from the experiments.

5.1 RQ1: How effective is a multi-agent workflow for clone detection and Extract Method refactoring in terms of correctness and usefulness?

5.1.1 Approach. To answer RQ1, we designed a controlled experiment using 60 manually created clone cases on the JHotDraw

Table 2: Full Multi-Agent Workflow Results (RQ1).

Model	Detected	Useful	Compilation	Test	Success
Claude Opus 4.6	60 (100.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)
Deepseek v3.2	60 (100.0%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	44 (73.3%)
GPT-5.2	60 (100.0%)	58 (96.7%)	58 (96.7%)	58 (96.7%)	58 (96.7%)
Gemini 3 Pro	60 (100.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)
Overall	240 (100.0%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	207 (86.3%)

Note: Detection is calculated over 60 cases per model. Compilation, Test, and Success are computed over cases without precondition violations.

Table 3: Iteration at which the workflow produced a useful refactoring

Model	1st iter.	2nd iter.	3rd iter.	Not useful
Claude Opus 4.6	41 (68.3%)	4 (6.7%)	3 (5.0%)	12 (20.0%)
Deepseek v3.2	32 (53.3%)	11 (18.3%)	1 (1.7%)	16 (26.7%)
GPT-5.2	47 (78.3%)	11 (18.3%)	0 (0.0%)	2 (3.3%)
Gemini 3 pro	46 (76.7%)	10 (16.7%)	1 (1.7%)	3 (5.0%)
Overall	166 (69.2%)	36 (15.0%)	5 (2.1%)	33 (13.8%)

Note. Percentages are calculated over 60 clone cases per model (240 cases overall). "Not useful" indicates that no iteration produced a refactoring satisfying the usefulness criteria.

project. As described in Section 4.8, we manually constructed the clones based on real methods from JHotDraw, which allowed us to control the clone type, location, and difficulty level.

We included three clone categories: Type-1 (exact copies), Type-2 (renamed identifiers or modified literals), and Type-3 (small insertions or deletions inside the method body). The clones were distributed across multiple classes to reduce bias from a specific context.

After preparing all clone cases, we applied the full multi-agent workflow to each case to perform clone detection and Extract Method refactoring. We set the maximum number of iterations to 3.

We evaluated effectiveness from two perspectives: correctness and usefulness. Correctness was measured by clone detection, successful compilation, and passing all tests. Usefulness was evaluated using our usefulness checker, which verifies whether the extracted method follows the structural definition adopted in this study.

To determine whether performance differences among models are statistically significant, we further conducted statistical analysis on the usefulness results. Since each clone case was evaluated by all models under the same experimental setting, we first applied Cochran's Q test to assess overall differences across related samples. When significant differences were detected, we performed pairwise McNemar tests with Bonferroni correction for multiple comparisons.

5.1.2 Results.

Detection. As shown in Table 2, the workflow achieved a 100.0% detection rate for all four models. Each model successfully identified all 60 clone cases, resulting in 240 out of 240 detections overall. This means that for every clone, the Detection Agent was able to locate the correct duplicated code region before the refactoring step.

Table 4: Refactoring success rate by clone type (20 pairs per type)

Model	Type-1	Type-2	Type-3	Overall
Claude Opus 4.6	20 (100%)	14 (70.0%)	14 (70.0%)	48 (80.0%)
DeepSeek v3.2	18 (90.0%)	15 (75.0%)	11 (55.0%)	44 (73.3%)
GPT-5.2	19 (95.0%)	19 (95.0%)	20 (100%)	58 (96.7%)
Gemini 3 Pro	20 (100%)	18 (90.0%)	19 (95.0%)	57 (95.0%)

Note. Percentages are calculated over 20 clone pairs per type for each model (60 cases per model in total).

This result is important because the later stages of the workflow depend on accurate detection. If the duplicated fragment is not correctly identified, the Extract Method refactoring may target the wrong lines or fail completely. In our experiment, we manually constructed Type-1, Type-2, and Type-3 clones, including cases with renamed variables, changed literal values, and small insertions or deletions. Even under these variations, the Detection Agent consistently returned the intended duplicated region. This suggests that the workflow is able to detect syntactic changes and structural differences.

Usefulness. Usefulness is the key factor that differentiates the models in our study. As shown in Table 2, GPT-5.2 achieves 58 out of 60 cases (96.7%) judged useful, and Gemini 3 Pro achieves 57 cases (95.0%). Claude Opus 4.6 achieves 48 useful cases (80.0%), and Deepseek v3.2 achieves 44 cases (73.3%). Overall, 207 out of 240 cases (86.3%) are considered useful. This shows that the workflow is able to produce meaningful Extract Method refactorings in most cases, but model choice still has a clear impact.

The iteration results in Table 3 help explain these differences. Models with higher usefulness also have more first-iteration successes. GPT-5.2 succeed for 47 cases (78.3%) and Gemini 3 Pro succeed in the first iteration for 46 cases (76.7%), while Deepseek v3.2 has only 32 first-iteration successes (53.3%) and a higher number of failed cases (16 cases, 26.7%). Claude Opus 4.6 lies in between. This suggests that models with stronger reasoning ability are more likely to generate a good extraction boundary and method structure in the first attempt, without relying heavily on feedback.

By reviewing the usefulness, we observed several common patterns among the non-useful cases. First, some extracted methods were too small and did not capture the whole cloned methods. In these cases, the extraction worked, but it did not improve code structure. Second, the models did not extract methods. Instead it directly removed the cloned method. Third, the cloned method body was replaced with the method call to the original method. Finally, there were cases where the models created multiple extracted methods. These issues did not meet the requirements of Extract Method refactoring, even if the code still works.

The iteration distribution in Table 3 shows that the usefulness checker and the feedback loop play an active role in improving the structural correctness of the refactoring. Many cases succeed in the first iteration, but a non-trivial portion needs additional rounds: 36 cases (15.0%) succeed in the second iteration, and 5 cases (2.1%) succeed in the third iteration. These later successes are not accidental. In our workflow, the usefulness checker provides concrete

feedback when a refactoring is not useful. The Refactoring Agent then uses this feedback to revise the extraction and try again. This process helps recover cases that would otherwise remain unsuccessful after the first attempt. The model-level results also support this: for both GPT-5.2 and Gemini 3 Pro, 11 (18.3%) and 10 (16.7%) cases, respectively, are improved to success in the second iteration, indicating that even strong models benefit from feedback. For Deepseek v3.2 and Claude Opus 4.6, the second and third iterations contribute even more relative gains, indicating that feedback is especially valuable when the first attempt is less stable. Overall, the feedback loop increases the final success rate by turning part of the initially unsuccessful outputs into useful refactorings, rather than relying only on first-try generation.

Overall, usefulness captures the practical value of the workflow. While detection is perfect and many cases succeed end-to-end, usefulness reveals whether the generated Extract Method truly improves the code in a meaningful way. In our design, usefulness is not only a final label but also an active signal in the workflow. The usefulness checker evaluates the extracted method and provides structured feedback when the refactoring is not useful. This feedback is then used by the Refactoring Agent in the next iteration. In this way, the feedback loop helps refine extraction boundaries.

Our results show that the combination of usefulness checking and iterative feedback increases the number of structurally valid Extract Method refactorings. At the same time, even within the same multi-agent framework and feedback mechanism, the final refactoring outcomes still vary across language models in terms of structural validity. This indicates that while the workflow structure provides guidance and correction, the underlying language model still plays a key role in determining how effectively the system can converge to a useful refactoring.

Listing 21: Example of Type-2 Clone with Modified Computation in scaledShape

```java
private static Shape scaledShape(Shape s, double scale) {
    if (s instanceof Rectangle) {
        Rectangle r = (Rectangle) s;
        return new Rectangle(
            (int) (r.x * scale),
            (int) (r.y * scale),
            (int) (r.width * scale),
            (int) (r.height * scale)
        );
    } else {
        throw new RuntimeException(
            "Cannot␣scale␣shape:␣" + s.getClass().getName()
        );
    }
}

private static Shape scaledShapeClone(Shape s, double scale)
      {
    if (s instanceof Rectangle) {
        Rectangle r = (Rectangle) s;
        return new Rectangle(
            (int) (r.x * scale) * (int) (r.x * scale),
            (int) (r.y * scale) * (int) (r.y * scale),
            (int) (r.width * scale),
            (int) (r.height * scale)
        );
    } else {
        throw new RuntimeException(
            "Cannot␣scale␣shape:␣" + s.getClass().getName()
        );
    }
}
```

Unuseful refactoring. Listing 21 illustrates a Type-2 clone. In Type-2 clones, the overall structure of the code is the same, but some identifiers, literals, or small expressions are modified. The two methods in Listing 21 have the same control structure and follow the same execution path. Both methods check whether the shape is a `Rectangle`, cast it, and return a new `Rectangle` object constructed from scaled coordinates. The `else` branch is also identical in both methods.

However, the computation inside the `return` statement is slightly different. In the original `scaledShape` method, each coordinate is multiplied by `scale` once. In `scaledShapeClone`, the expressions for `r.x` and `r.y` are multiplied twice: `(int) (r.x * scale) * (int) (r.x * scale)` and `(int) (r.y * scale) * (int) (r.y * scale)`. The width and height calculations remain the same. This means that the method bodies are structurally identical and that only the internal arithmetic expressions are changed. According to the standard definition, this satisfies the criteria of a Type-2 clone.

For Deepseek v3.2, Claude Opus 4.6, and Gemini 3 Pro, all three attempts failed the usefulness check. In every attempt, the generated refactoring was labeled as incomplete refactoring. The models either extracted only part of the duplicated logic or left duplicated computations in the original methods. As a result, the duplication was not fully removed, and the transformation did not strictly follow the Extract Method definition used in this study.

In contrast, GPT-5.2 successfully passed the usefulness check on this example. It correctly identified the full duplicated region and extracted it into a shared method. The final refactoring reduced duplication and met our structural criteria. This case shows that even when compilation and testing are not the main issue, structural completeness remains challenging for several models. It also highlights that stronger models can better reason about extraction boundaries under strict usefulness constraints.

Compilation. Table 2 shows that 207 out of 240 cases (86.3%) compile successfully. At the model level, GPT-5.2 achieves 58 successful cases (96.7%), Gemini 3 Pro achieves 57 cases (95.0%), followed by Claude Opus 4.6 with 48 cases (80.0%) and Deepseek v3.2 with 44 cases (73.3%).

However, in our workflow, compilation is executed after the usefulness checker accepts the refactoring. This means compilation is not the primary filtering step. Instead, it confirms that the final accepted refactoring can be integrated into the project without breaking the build. The fact that the compilation numbers are identical to usefulness and success indicates that once a refactoring passes the usefulness evaluation, it almost always compiles successfully. This also suggests that the LLM-generated code is generally syntactically valid and structurally consistent.

Testing. The testing results follow the same pattern as compilation, with 207 out of 240 cases (86.3%) passing all tests. The

model-level distribution is identical: 58 cases (96.7%) for GPT-5.2, 57 cases (95.0%) for Gemini 3 Pro, 48 cases (80.0%) for Claude Opus 4.6, and 44 cases (73.3%) for Deepseek v3.2.

Since testing is performed after usefulness checking and successful compilation, the test results confirm behavioral preservation for the final accepted refactorings. The identical numbers across Useful, Compilation, Test, and Success indicate that the main filtering mechanism in our pipeline is the usefulness checker. In other words, most LLM outputs are able to compile and pass tests, but only those that satisfy our structural validity criteria are accepted as final successes.

Impact of Clone Type. Table 4 further breaks down the success rate by clone type. We constructed 20 clone pairs for each type (Type-1, Type-2, and Type-3), allowing us to analyze how structural complexity affects refactoring performance.

For Type-1 clones, all models achieve high success rates. Claude Opus 4.6 and Gemini 3 Pro reach 100%, GPT-5.2 achieves 95.0%, and Deepseek v3.2 achieves 90.0%. This indicates that exact duplicates are relatively easy for all models to refactor correctly.

For Type-2 clones, which involve renamed variables or modified literals, performance varies more clearly across models. GPT-5.2 achieves 95.0% and Gemini 3 Pro achieves 90.0%, while Claude Opus 4.6 and Deepseek v3.2 achieve 70.0% and 75.0%, respectively. This suggests that handling renamed identifiers and minor syntactic changes requires stronger contextual reasoning.

Type-3 clones, which contain small insertions or deletions inside the method body, show the largest performance gap. GPT-5.2 achieves 100% and Gemini 3 Pro achieves 95.0%, while Claude Opus 4.6 and Deepseek v3.2 drop to 70.0% and 55.0%, respectively. These results indicate that structural differences within the method body significantly increase refactoring difficulty, especially for weaker models.

Overall, the results suggest that clone complexity has a measurable impact on Extract Method success. Exact duplicates are easiest to refactor, while Type-3 clones require more precise reasoning about extraction boundaries. Stronger models remain stable across clone types, whereas weaker models show sharper performance degradation as structural complexity increases.

Statistical Analysis. Table 5 reports the statistical comparison of model success rates. We first applied Cochran's Q test to examine whether there is an overall difference among the four models. The result is significant ($p = 6.7 \times 10^{-5}$), which suggests that at least one model performs differently from the others.

To further understand where the differences lie, we conducted pairwise McNemar tests with Bonferroni correction ($\alpha = 0.0083$). Among the six pairwise comparisons, only the differences between DeepSeek and GPT-5.2 ($p = 0.00131$), and between DeepSeek and Gemini 3 Pro ($p = 0.00098$) remain statistically significant after correction.

The other comparisons do not reach statistical significance under the adjusted threshold. In particular, although GPT-5.2 and Gemini 3 Pro show slightly higher success rates than Claude Opus 4.6, these differences are not statistically significant after Bonferroni correction. Likewise, there is no significant difference between GPT-5.2 and Gemini 3 Pro.

Table 5: Statistical comparison of model success rates (RQ1).

Comparison	b	c	p-value	Significant (Bonf.)
Cochran's Q overall test				
All models	–	–	6.7×10^{-5}	Yes
Pairwise McNemar tests ($\alpha = 0.0083$)				
Claude vs DeepSeek	9	5	0.424	No
Claude vs GPT-5.2	2	12	0.0129	No
Claude vs Gemini 3 Pro	1	10	0.0117	No
DeepSeek vs GPT-5.2	2	16	0.00131	**Yes**
DeepSeek vs Gemini 3 Pro	1	14	0.00098	**Yes**
GPT-5.2 vs Gemini 3 Pro	3	2	1.000	No

Note: b denotes the number of cases where Model A succeeds and Model B fails; c denotes the number of cases where Model B succeeds and Model A fails. Bonferroni correction was applied for six pairwise comparisons ($\alpha = 0.05/6 = 0.0083$).

Overall, the statistical results indicate that DeepSeek v3.2 performs significantly worse than GPT-5.2 and Gemini 3 Pro under our experimental setting. For the other model pairs, the observed performance differences may reflect practical variation, but they are not strong enough to conclude statistical significance under the corrected threshold.

RQ1 Summary.
The multi-agent workflow achieves perfect clone detection (100%) and produces useful Extract Method refactorings in 207 out of 240 cases (86.3%) overall. GPT-5.2 achieves the highest usefulness rate (58/60, 96.7%), followed by Gemini 3 Pro (57/60, 95.0%), while Claude Opus 4.6 and Deepseek v3.2 achieve 80.0% and 73.3%, respectively. Iterative refinement further improves outcomes, with 36 additional successes in the second iteration and 5 in the third. Cochran's Q test indicates a significant overall difference among models ($p = 6.7 \times 10^{-5}$), and pairwise McNemar tests show that Deepseek v3.2 performs significantly worse than GPT-5.2 and Gemini 3 Pro.

5.2 RQ2: What is the contribution of each component of the multi-agent framework?

5.2.1 Approach. To answer RQ2, we conduct an ablation study to understand the contribution of each major component in our multi-agent framework. The goal is to isolate the effect of individual components and measure how much each contributes to the final refactoring success rate.

We use the same 60 manually constructed clone cases on the JHotDraw project described in RQ1. We also use the same four language models: GPT-5.2, Gemini 3 Pro, Claude Opus 4.6, and Deepseek v3.2. The dataset, evaluation criteria, and maximum number of attempts remain the same. Only the workflow configuration is changed.

We design four ablation settings:

(1) **No RAG**: We remove retrieval-augmented generation from both the Detection Agent and the Refactoring Agent. The models no longer receive example-based guidance in the prompt.

Table 6: RQ2 ablation study comparing the full workflow (with RAG) and the workflow without RAG.

	Full workflow (with RAG)					Ablation: No RAG				
Model	**Detected**	**Useful**	**Compilation**	**Test**	**Success**	**Detected**	**Useful**	**Compilation**	**Test**	**Success**
Claude Opus 4.6	60 (100.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	60 (100.0%)	47 (78.3%)	47 (78.3%)	47 (78.3%)	47 (78.3%)
Deepseek v3.2	60 (100.0%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	60 (100.0%)	20 (33.3%)	20 (33.3%)	20 (33.3%)	20 (33.3%)
GPT-5.2	60 (100.0%)	58 (96.7%)	58 (96.7%)	58 (96.7%)	**58 (96.7%)**	59 (98.3%)	56 (93.3%)	56 (93.3%)	56 (93.3%)	**56 (93.3%)**
Gemini 3 Pro	60 (100.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	60 (100.0%)	51 (85.0%)	51 (85.0%)	51 (85.0%)	51 (85.0%)
Overall	240 (100.0%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	239 (99.6%)	174 (72.5%)	174 (72.5%)	174 (72.5%)	174 (72.5%)

Note. Each model is evaluated on 60 clone cases (240 total). Percentages are computed over 60 cases per model and over 240 cases for Overall.

(2) **No Usefulness Checker**: We disable the usefulness checker. Refactorings are evaluated only by compilation and testing, without structural validity checking.

(3) **No Feedback Loop**: We remove the iterative feedback mechanism. Each model generates a refactoring only once, without receiving compile errors, test failures, or usefulness feedback for correction.

For each setting, we run all four models in the same 60 clone cases. We measure detection rate, compilation success, test passing rate, and final usefulness-based success when applicable. By comparing these ablated variants with the full multi-agent workflow, we can quantify how much each component contributes to improving structural correctness and overall refactoring success.

5.2.2 *Results.*

No RAG. Table 6 shows an in-depth comparison of the complete workflow with the inclusion of the RAG mechanism and the ablated version of the workflow with the removal of the RAG mechanism. We can note that the overall performance of the detection mechanism remains almost the same. In the complete workflow, all four models have 100% detection accuracy (i.e., all 240 cases are detected). However, with the removal of the RAG mechanism, the overall detection accuracy drops from 240 to 239 cases, which means that one case is not detected. This shows that the examples generated using the retrieval mechanism are not critical for the clone localization problem. The Detection Agent seems to be strong enough to locate the duplicated regions.

However, the situation is very different when we look at usefulness and overall success. In the complete workflow, the overall success rate is 86.3% (207/240 cases). If we remove RAG from the workflow, the overall success rate drops to 72.5% (174/240 cases), a drop of 13.8% in overall success rate. This is a significant drop, and it indicates that, although the model is still able to detect clones, it is having difficulty generating structurally valid Extract Method refactoring operations without the aid of example-based constraints. Note that usefulness, compilation, testing, and overall success are all correlated in our workflow, and so we are seeing a drop in structural validity rather than syntax or behavioral correctness.

The effect of RAG removal on these models varies, but for GPT-5.2, the drop from 96.7% to 93.3% is relatively small. For Gemini 3 Pro, the drop from 95.0% to 85.0% is larger. For Claude Opus 4.6, the drop from 80.0% to 78.3% is minimal. However, for Deepseek v3.2, the drop from 73.3% to 33.3% is dramatic, implying that more than half of its successful attempts will fail to refactor correctly in the absence of retrieval support. This implies that Deepseek heavily depends on example-based patterns to ensure proper extraction boundary and structural consistencies in its refactoring attempts.

By providing actual examples, RAG helps the model apply the proper and consistent Extract Method pattern. This effect is more important for models that tend to be less stable in their reasoning for clone refactoring. While models like GPT-5.2 tend to be more robust, even they benefit from the support of RAG. RAG plays the role of a structural stabilizer in the multi-agent workflow. It helps prevent incomplete and improper refactoring and improves the final success rate.

No Usefulness Checker. Table 7 shows a comparison between the entire workflow and an ablated version where we remove the usefulness checker. The most obvious thing to notice is that detection is unchanged. All four models still detect clones in 60/60 cases. Hence, overall detection is still 240/240 (100.0%).

However, usefulness and success rates are reduced. For the entire workflow, overall success rate is 86.3% (207/240). When we remove the usefulness checker, overall success rate drops to 66.3% (159/240). Hence, we have a reduction of 20.0%. This is a larger reduction than in the No RAG setting. The result suggests that the usefulness checker plays an important role in maintaining structural validity during refactoring.

An important observation is that in the ablated case, the columns *Useful*, *Compilation*, *Test*, and *Success* are identical in each model. This implies that the majority of failed cases are not driven by syntax or test issues, but rather by structural issues related to the definition of the Extract Method refactoring. When the usefulness checker is no longer part of the workflow, there is no mechanism to eliminate structurally weak refactoring operations, even if the refactored code is syntactically correct and passes all tests.

In all models, the percentage is reduced, and we observe the following changes: Claude Opus 4.6: from 80.0% to 65.0% (reduced by 15.0 points); Deepseek v3.2: from 73.3% to 50.0% (reduced by 23.3 points); GPT-5.2: from 96.7% to 70.0% (reduced by 26.7 points); Gemini 3 Pro: from 95.0% to 80.0% (reduced by 15.0 points).

Overall, these results suggest that the usefulness checker is a significant contributor to workflow effectiveness. Removing it results in a noticeable decrease in the number of useful and successful refactorings, even though detection is still perfect and compilation rates remain high. This again confirms our observation that structural correctness is a key challenge for clone refactoring with LLM.

Table 7: RQ2 ablation study: comparison between the full multi-agent workflow and the workflow without the usefulness checker.

	Full workflow (with usefulness checker)					Ablation: No Usefulness Checker				
Model	**Detected**	**Useful**	**Compilation**	**Test**	**Success**	**Detected**	**Useful**	**Compilation**	**Test**	**Success**
Claude Opus 4.6	60 (100.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	60 (100.0%)	39 (65.0%)	39 (65.0%)	39 (65.0%)	39 (65.0%)
Deepseek v3.2	60 (100.0%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	60 (100.0%)	30 (50.0%)	30 (50.0%)	30 (50.0%)	30 (50.0%)
GPT-5.2	60 (100.0%)	58 (96.7%)	58 (96.7%)	58 (96.7%)	**58 (96.7%)**	60 (100.0%)	42 (70.0%)	42 (70.0%)	42 (70.0%)	42 (70.0%)
Gemini 3 Pro	60 (100.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	60 (100.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	**48 (80.0%)**
Overall	240 (100.0%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	240 (100.0%)	159 (66.3%)	159 (66.3%)	159 (66.3%)	159 (66.3%)

Note: Each model is evaluated on 60 clone cases (240 total). Percentages are computed over 60 cases per model and 240 cases overall.

Table 8: RQ2 ablation study: comparison between the full multi-agent workflow and the workflow without the feedback loop.

	Full workflow					Ablation: No Feedback Loop				
Model	**Detected**	**Useful**	**Compilation**	**Test**	**Success**	**Detected**	**Useful**	**Compilation**	**Test**	**Success**
Claude Opus 4.6	60 (100.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	48 (80.0%)	60 (100.0%)	41 (68.3%)	41 (68.3%)	41 (68.3%)	41 (68.3%)
Deepseek v3.2	60 (100.0%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	44 (73.3%)	60 (100.0%)	21 (35.0%)	21 (35.0%)	21 (35.0%)	21 (35.0%)
GPT-5.2	60 (100.0%)	58 (96.7%)	58 (96.7%)	58 (96.7%)	**58 (96.7%)**	60 (100.0%)	52 (86.7%)	52 (86.7%)	52 (86.7%)	**52 (86.7%)**
Gemini 3 Pro	60 (100.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	57 (95.0%)	60 (100.0%)	50 (83.3%)	50 (83.3%)	50 (83.3%)	50 (83.3%)
Overall	240 (100.0%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	207 (86.3%)	240 (100.0%)	164 (68.3%)	164 (68.3%)	164 (68.3%)	164 (68.3%)

Note: Each model is evaluated on 60 clone cases (240 total). Percentages are computed over 60 cases per model and 240 cases overall.

No Feedback Loop. Table 8 presents a comparison between the complete multi-agent workflow and the ablated workflow where we did not include the feedback loop between iterations. The first observation is that the detection part is still the same. The four models still detect clones in 60/60 instances, and hence the overall detection is still at 240/240 (100.0%). This is because the feedback loop is only important in the refactoring refinement part, not in the clone localization part.

The significant changes are in the overall usefulness and, in turn, overall success. The overall success rate in the complete workflow is 86.3% or 207/240. In the ablated workflow, the overall success is only 68.3% or 164/240, a drop of 18.0% from the complete workflow.

An interesting observation of this table is that for each model, columns *Useful*, *Compilation*, *Test*, and *Success* always remain the same. This indicates that the reduced success rate is not because of compilation or test failures. Most of these failures are structurally invalid refactorings that don't meet our criteria for an extract method. In a normal workflow, this feedback loop allows the model to correct its output if it is structurally incorrect after receiving a validation signal. Without this feedback loop, these structurally incorrect initial attempts become the final output.

As can be seen, each model has a different level of impact. The model that is impacted the most is Deepseek v3.2. The success rate is reduced by 38.3 percentage points, from 73.3% to 35.0% or 44 to 21 cases. Claude Opus 4.6 has a reduced success rate by 11.7 percentage points, from 80.0% to 68.3% or 48 to 41 cases. The Gemini 3 Pro model has a reduced success rate by 11.7 percentage points, from 95.0% to 83.3% or 57 to 50 cases. GPT-5.2 still has the highest success rate and is impacted by 10.0 percentage points, from 96.7% to 86.7% or 58 to 52 cases.

Overall, this ablation verifies that the feedback loop is a significant stabilizing factor for this workflow. While it has no impact on detection, it greatly improves the number of structurally correct and useful Extract Method refactorings by allowing for an iterative correction of borderline or incomplete results.

Listing 22: Example of Unchanged Cloned Methods

```
public void newView() {
    if (!view().isInteractive()) {
        return;
    }

    // create new window with view to an existing drawing
    newWindow(view().drawing());

    String copyTitle = getDrawingTitle();
    if (copyTitle != null) {
        setDrawingTitle(copyTitle);
    }
    else {
        setDrawingTitle(getDefaultDrawingTitle());
    }
}

public void newView_clone() {
    if (!(this.view().isInteractive())) {
        return;
    }

    // create new window with view to an existing drawing
    this.newWindow((this.view().drawing()));
```

```
    String copyTitle = String.valueOf(this.getDrawingTitle()
     );
    if ((copyTitle) != null) {
        this.setDrawingTitle((copyTitle));
    }
    else {
        this.setDrawingTitle((this.getDefaultDrawingTitle())
     );
    }
}
```

New Observation of Failed Patterns. In addition to the failed Extract Method patterns identified in prior work [35] (such as incomplete extraction or direct clone removal), we observed another failure case shown in Listing 22. In this case, the refactoring output leaves both the original method and the cloned method completely unchanged. No extraction is performed, and the duplicated logic remains intact. This behavior was observed in the ablation study when using the Deepseek v3.2 model. Without structural guidance and iterative feedback, the model sometimes produces a no-op result instead of attempting a refactoring. Although the code still compiles and passes tests, the duplication is not removed and therefore the refactoring cannot be considered successful. This pattern indicates that, without structural validation and iterative feedback, some models may fail to perform any modification, even when clones are clearly present. In such cases, the refactoring stage leaves the code unchanged instead of producing a correct transformation.

Overall Analysis of RQ2. Figure 2 gives a clear visual summary of all ablation settings for RQ2. As we can observe, if we compare the bars of the different models, one observation is immediately clear: detection is always stable across all settings, while usefulness and final success vary across settings after the removal of structural support components. This again confirms that clone localization is not the problem, but rather the challenge is applying structurally correct Extract Method refactoring.

As we can observe across all procedures, the whole workflow of the multi-agent system always has the highest success rate of 86.3%. The removal of RAG results in a success rate of 72.5%. The removal of the usefulness checker results in a success rate of 66.3%. The removal of the feedback loop results in a success rate of 68.3%.

Of all the components, the usefulness checker has the largest individual impact. The decrease in performance when the usefulness checker is disabled (−20.0 percentage points) is greater in absolute value than disabling RAG (−13.8 percentage points) or disabling the feedback loop (−18.0 percentage points). This indicates that structural validation is a key component in achieving structurally valid Extract Method refactoring.

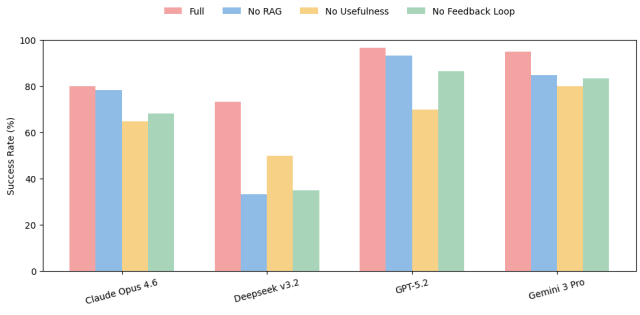

Figure 2: RQ2 ablation study success rate per model.

While compilation and testing are important for behavioral correctness, usefulness checking is important for structural correctness, and structural correctness is usually the dominant source of failure in LLM-based refactoring.

While RAG and usefulness checking are related, they are distinct in that RAG guides generation towards structurally correct Extract Method patterns, and usefulness checking filters out structurally incorrect transformations. The feedback loop is important for stability, allowing for iterative correction of borderline cases.

In conclusion, the result of RQ2 proves that structural enforcement mechanisms play an important role in the guarantee of LLM-based clone refactoring. Detection alone is not enough, nor is behavioral validation alone. High success rates can only be achieved by combining retrieval guidance, structural usefulness checks, and iterative feedback.

RQ2 Summary.
In all cases of ablation, the stability of the detection result remains the same (100%), which suggests that the localization of the clone is not the key problem. Instead, the differences in performance are mainly related to the structural validation and refinement.
If the RAG is removed, the overall success rate drops from 86.3% to 72.5% (−13.8 percentage points). When the feedback loop is eliminated, the result drops to 68.3% (−18.0 percentage points). Removing the usefulness checker has the most significant impact on the overall success rate, which drops to 66.3% (−20.0 percentage points).
For different models, the lower-performing models, such as Deepseek v3.2, are more sensitive to the removal of components, while the better-performing models, such as GPT-5.2, are more stable but can still benefit from structural guidance. In general, the high reliability of refactoring can only be ensured with the collaborative contributions of retrieval guidance, usefulness validation, and iterative feedback.

5.3 RQ3: How much do usefulness, compilation and test feedback improve refactoring success compared to a single-LLM setting?

5.3.1 Approach. To answer RQ3, we compare our multi-agent workflow with a single-LLM baseline setting and traditional tool JDeodorant. The goal of this comparison is to measure how much usefulness checking, compilation feedback, and testing feedback improve the final refactoring success rate.

We used the same 60 manually constructed clone cases on the JHotDraw project as described in RQ1. We also use the same four language models: GPT-5.2, Gemini 3 Pro, Claude Opus 4.6, and Deepseek v3.2. This ensures that the input tasks, project context, and evaluation criteria remain identical across both settings. The only difference lies in the workflow design.

Table 9: Effectiveness of the single-LLM setting without usefulness and feedback loop.

Model	Detected	Useful	Compilation	Test	Success
Claude Opus 4.6	60 (100.0%)	38 (63.3%)	60 (100.0%)	60 (100.0%)	38 (63.3%)
Deepseek v3.2	60 (100.0%)	42 (70.0%)	60 (100.0%)	60 (100.0%)	42 (70.0%)
GPT-5.2	60 (100.0%)	53 (88.3%)	60 (100.0%)	60 (100.0%)	53 (88.3%)
gemini 3 pro	60 (100.0%)	38 (63.3%)	60 (100.0%)	60 (100.0%)	38 (63.3%)
Overall	240 (100.0%)	171 (71.3%)	240 (100.0%)	240 (100.0%)	171 (71.3%)

Note: Success means detected, compiled, passed all tests, and satisfied the usefulness criterion.

Table 10: Refactoring Success Rate by Clone Type – Single LLM

Model	Type-1	Type-2	Type-3	Overall
Claude Opus 4.6	16 (80.0%)	13 (65.0%)	9 (45.0%)	38 (63.3%)
DeepSeek v3.2	17 (85.0%)	14 (70.0%)	11 (55.0%)	42 (70.0%)
GPT-5.2	18 (90.0%)	19 (95.0%)	16 (80.0%)	53 (88.3%)
Gemini 3 Pro	13 (65.0%)	10 (50.0%)	15 (75.0%)	38 (63.3%)

In the single-LLM setting, we remove the usefulness checker, compilation feedback, and testing feedback loop. The model receives the original clone case and is prompted to perform Extract Method refactoring in one step. The generated refactoring is directly applied to the project without structured feedback from other agents. In other words, the model does not receive corrective signals about the usefulness or validity of the refactoring during the process.

For fairness, we allow up to three independent attempts for each case, which matches the maximum iteration number used in the multi-agent workflow. If a refactoring attempt fails to meet the success criteria, we re-run the model with the same prompt and input, without providing feedback from previous attempts. We record the first successful attempt within these three runs. If all three attempts fail, the case is marked as unsuccessful.

We evaluate effectiveness using the same success definition as in RQ1. A case is considered successful only if the refactoring compiles, passes all tests, and satisfies the usefulness criterion. By keeping the dataset, models, success criteria, and maximum number of attempts consistent, this design isolates the effect of the feedback mechanisms. This allows us to directly measure how much structured usefulness checking and iterative feedback contribute to improving refactoring outcomes compared to a single-LLM generation process.

5.3.2 *Results.*

Detection. As shown in Table 9, the single-LLM setting achieves a 100.0% detection rate. Each model successfully detected all 60 clone cases in this experiment, resulting in 240 out of 240 detections overall. This indicates that removing the usefulness checker and feedback loop does not affect the ability of the workflow to locate duplicated code fragments.

Similar to RQ1, the clone cases include Type-1, Type-2, and Type-3 variants with renamed variables and small structural changes.

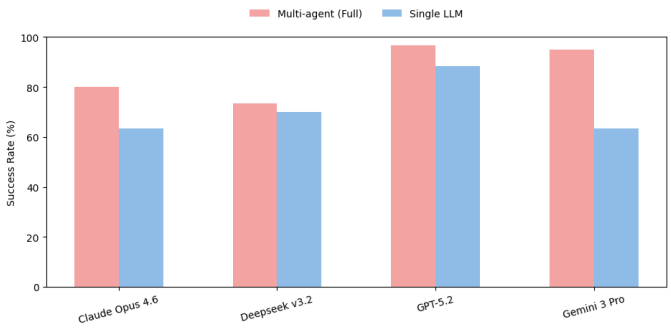

Figure 3: Comparison of refactoring success rate between the multi-agent workflow and the single-LLM setting.

Even without structured feedback, the detection component consistently identifies the intended duplicated regions. This suggests that clone detection in our setup is stable and does not rely on iterative correction.

Therefore, detection is not a differentiating factor between the multi-agent workflow and the single-LLM setting. The main differences between the two settings appear in the refactoring stage, rather than in clone localization.

Usefulness. Table 9 shows that in the single-LLM setting, 171 out of 240 cases (71.3%) are judged useful. In contrast, the multi-agent workflow achieves 206 out of 240 useful cases (85.8%), corresponding to an overall improvement of 14.5 percentage points when usefulness checking and feedback loops are enabled.

The improvement differs across models. For GPT-5.2, usefulness increases from 88.3% (53/60) in the single-LLM setting to 96.7% in the multi-agent workflow. For Deepseek v3.2, usefulness increases from 70.0% (42/60) to 73.3%. Claude Opus 4.6 shows a substantial gain, from 63.3% (38/60) to 80.0%. Gemini 3 Pro also improves markedly, from 63.3% (38/60) to 95.0%. These differences are consistent with Figure 3, where the performance gap between the single-LLM and multi-agent settings is smallest for GPT-5.2 and largest for Gemini 3 Pro.

The results suggest that usefulness checking and iterative feedback help correct structural mistakes in the generated refactorings. In the single-LLM setting, many unsuccessful cases correspond to incomplete refactoring, direct clone removal, call-based substitution, or delegation. Although such outputs often preserve behavior and pass compilation and testing, they do not strictly satisfy the Extract Method definition adopted in this study. Without feedback, these structural deviations remain uncorrected. In the multi-agent workflow, however, the usefulness checker detects these violations and triggers revision. As a result, several structurally flawed first attempts are refined into acceptable refactorings.

Overall, the comparison shows that usefulness feedback does not merely filter outputs. It increases the rate of structurally valid Extract Method refactorings by guiding the model toward correct extraction boundaries. The effect is particularly pronounced for Claude Opus 4.6 and Gemini 3 Pro, where large gains indicate stronger dependence on structural guidance.

Compilation. In the single LLM scenario, all the refactorings compile successfully. This is clear from Table 9, where the compilation for all the models is at 100.0% (240/240). This suggests that the LLMs are generally capable of producing syntactically correct Java code.

Unlike the multi-agent scenario, the compilation here is not for correction. Rather, it is for ensuring that the generated code can be feasibly incorporated. Therefore, the fact that all the refactorings compile successfully suggests that the issue of syntax and type consistency is not the main problem here. Rather, the problem is ensuring the structural correctness of the refactorings, as reflected by the lower usefulness.

Testing. The results of the testing for the single LLM scenario remain 100% for all models, where all 240 refactoring operations pass the test suite. This indicates that the behavior of the program is preserved for the given test suite.

Nonetheless, it is essential to note that passing the test suite does not necessarily mean that the refactoring operation is structurally correct, since there are numerous structurally incorrect outputs that also passed the test suite due to their behavior equivalence. Therefore, this validates the necessity of the usefulness check, where behavior-preserving but structurally incorrect refactoring operations can be distinguished from the correct Extract Method refactoring operations.

Impact of Clone Type (Single-LLM). Table 10 shows the refactoring success rate for different clone types in the single-LLM setting. Similar to the analysis of RQ1, we analyze Type-1, Type-2, and Type-3 clone types individually. This is to understand the impact of structural complexity on the performance of the LLMs in the absence of structured feedback.

For Type-1 clone type, where the clones are exact duplicates, all LLMs have high refactoring success rates. GPT-5.2 outperforms the others with the highest success rate of 90.0%, followed by DeepSeek v3.2 with an 85.0% success rate, and then Claude Opus 4.6 with an 80.0% success rate. However, the performance of Gemini 3 Pro drops significantly with a mere 65.0% success rate.

For Type-2 clone type, where the variables are renamed or the literals are changed, the performance of the LLMs is more distinguishable. Again, GPT-5.2 performs the best with an outstanding success rate of 94.4%, while DeepSeek v3.2 and Claude Opus 4.6 have lower success rates of 66.7% and 61.1%, respectively. In contrast, the performance of Gemini 3 Pro drops significantly with a mere 44.4% success rate.

Type-3 clones are the most challenging. These clones contain small insertions or deletions inside the method body, which require more precise extraction boundaries. GPT-5.2 still achieves 81.2%, but Claude Opus 4.6 and DeepSeek v3.2 drop to 43.8% and 50.0%, respectively. Gemini 3 Pro achieves 68.8%, which is higher than its Type-2 performance but still lower than GPT-5.2. The performance gap across models is widest for Type-3 clones, indicating that structural variations significantly increase refactoring difficulty in the absence of iterative feedback.

Overall, the single-LLM results show a clearer degradation trend from Type-1 to Type-3 clones compared to the multi-agent setting. Without usefulness checking and revision, models are less able to handle structural differences inside method bodies. This reinforces the role of iterative feedback in stabilizing performance across clone types.

Table 11: Effectiveness of traditional tools JDeodorant

Tool	Deckard Detected	Precondition	Compilation	Test	Useful
JDeodorant	41 (68.3%)	6 (14.6%)	34 (97.1%)	34 (97.1%)	34 (97.1%)

Note. Precondition refers to cases where JDeodorant did not apply the Extract Method refactoring because of precondition violation. The percentage is calculated on the precondition cases found (41). The percentage for Compilation, Test, and Useful is calculated on cases without precondition violation.

Traditional Tool. Based on Table 11, JDeodorant was applied to the clone pairs detected by Deckard [21]. Out of the 60 clone cases, Deckard detected 41 cases (68.3%). Among these detected cases, JDeodorant reported 6 precondition violations (14.6%). For the remaining 35 cases without precondition violations, the tool successfully applied Extract Method refactoring on 34 cases. All generated refactorings compiled and passed the test suite (97.1% when calculated over non-precondition cases). These 34 cases were also considered useful according to our usefulness criterion.

When comparing JDeodorant with the multi-agent workflow, JDeodorant applies Extract Method when its precondition checks are satisfied. Therefore, its success rate is high on the applicable subset (34/35, 97.1%) because 19 cases are not detected by Deckard and 6 detected cases are blocked by precondition violations. In contrast, the multi-agent workflow attempts refactoring for all clone cases and reports an overall end-to-end success rate of 85.8% over the full set.

When applying JDeodorant to this clone pair, the tool reports a precondition violation. When JDeodorant reports a precondition violation in this example, it reflects the safety constraints enforced by its Extract Method preconditions.

Precondition Violation 1. In the clone pair shown in Listing 23, the second method introduces an intermediate variable, `DragSourceContext ctx`, which participates in data-flow relationships across multiple statements. When JDeodorant analyzes a candidate extraction region, it checks whether all variables used inside the region can be cleanly parameterized or returned without breaking dependencies. In this case, the variable `ctx` has dependencies both inside and outside the potential extraction boundary. This means that extracting the duplicated logic into a shared helper method would require restructuring the data flow in a way that violates JDeodorant's correctness constraints.

Listing 23: Example of Type-3 Clone With Precondition Violation

```
public void dragEnter(DragSourceDragEvent dsde) {
    log("DragSourceDragEvent-dragEnter");
    if (autoscrollState == null) {
Component c = dsde.getDragSourceContext().getComponent();
        if (JComponent.class.isInstance(c)) {
            JComponent jc = (JComponent) c;
            autoscrollState = new Boolean(jc.getAutoscrolls
    ());
            jc.setAutoscrolls(false);
        }
    }
```

```
}

public void dragEnterClone(DragSourceDragEvent dsde) {
    log("DragSourceDragEvent-dragEnter");
    if (autoscrollState == null) {

      DragSourceContext ctx = dsde.getDragSourceContext();
          Component c = ctx.getComponent();
        if (c instanceof JComponent) {
            JComponent jc = (JComponent) c;
            autoscrollState = Boolean.valueOf(jc.
     getAutoscrolls());
            jc.setAutoscrolls(false);
        }
    }
}
```

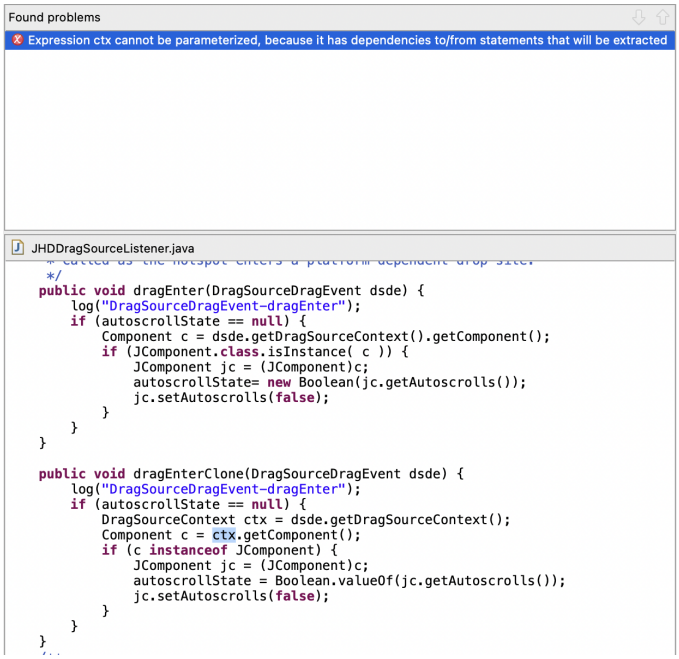

Figure 4: Precondition Violation: Dependency

Therefore, as shown in Figure 4, the precondition violation indicates that, under JDeodorant's safety rules, the refactoring cannot be performed without potentially changing behavior.

Precondition Violation 2. The clone pair shown in Listing 24 is a typical Type-2 clone. The two methods share the same control flow and logic. Both methods compute the image width and height, obtain the size of the view, and then use nested while loops to repeatedly draw the image. The overall structure of the two methods is identical.

Listing 24: Example of Type-2 Clone With Precondition Violation.

```
private void drawPattern(Graphics g, Image image,
     DrawingView view) {
    int iwidth = image.getWidth(view);
    int iheight = image.getHeight(view);
    Dimension d = view.getSize();
    int x = 0;
    int y = 0;

    while (y < d.height) {
        while (x < d.width) {
            g.drawImage(image, x, y, view);
            x += iwidth;
        }
        y += iheight;
        x = 0;
    }
}

private void drawPattern_Cloned(Graphics g, Image image,
     DrawingView view) {
    int iwidth = image.getWidth(view);
    int iheight = image.getHeight(view);
    Dimension d = view.getSize();
    double x = 0;
    double y = 0;

    while (y < d.height) {
        while (x < d.width) {
            g.drawImage(image, (int)x, (int)y, view);
            x += iwidth;
        }
        y += iheight;
        x = 0;
    }
}
```

The only difference lies in the type of the local variables `x` and `y`. In the first method, both variables are declared as `int`. In the second method, they are declared as `double`. Although this difference appears minor, it changes the numeric type used in the loop and during arithmetic operations. In the second method, explicit casting to `int` is required when calling `drawImage`.

When JDeodorant analyzes this clone pair, it checks whether the duplicated region can be safely extracted into a shared method. However, the variable types do not match across the two methods. According to the tool's precondition rules, inconsistent variable types prevent a direct extraction. As a result shown in Figure 5, JDeodorant reports the following precondition violation: *Type int of variable members does not match with type double of variable members.*

RQ3 Summary. Removing usefulness checking and iterative feedback of compilation and testing agents reduces the overall refactoring success rate from 85.8% (multi-agent) to 70.4% (single-LLM), a drop of 15.4%.
In the single-LLM setting, all generated refactorings compile and pass tests (100%), indicating that syntax and behavioral preservation are not the main challenges. However, many outputs fail to meet structural Extract Method requirements due to incomplete refactoring, direct clone removal, or delegation-style transformations.
The multi-agent workflow improves success primarily by correcting structural mistakes through usefulness evaluation and iterative revision. The improvement is especially significant for Claude Opus 4.6 and Gemini 3 Pro, demonstrating that structured feedback plays a crucial role in guiding models.

6 DISCUSSION

6.1 Structural Correctness vs. Behavioral Correctness

The results reveal that there is a significant difference between behavioral correctness and structural correctness. In the single LLM case, all the generated refactoring code compiles correctly and has passed the test cases, but only 70.4% of the code has achieved the definition of Extract Method. This implies that it is not guaranteed that the code behavior is preserved to correctly remove the code clones.

In some cases of incorrectness, the generated code has preserved the program's functionality but has failed to correctly perform the Extract Method refactoring. For example, refactorings such as direct code clone removal, or incomplete refactoring can be syntactically correct and pass all the test cases, but they have not achieved the goal of the Extract Method refactoring.

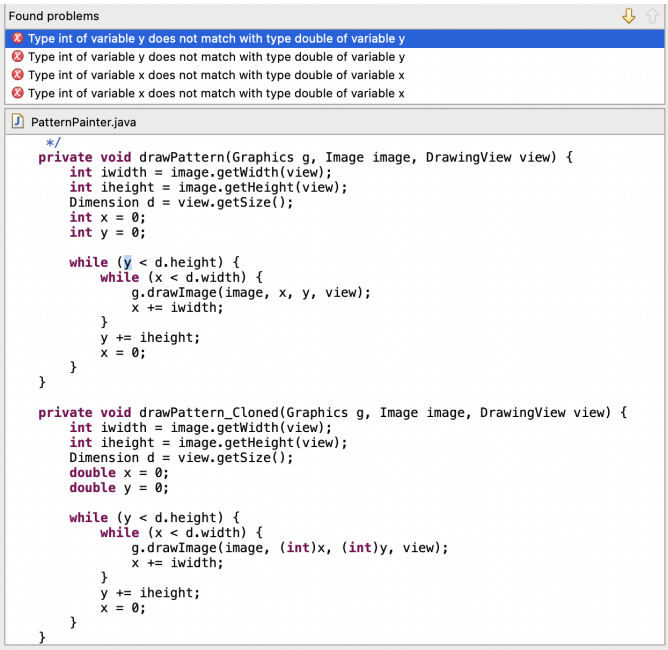

Figure 5: Precondition Violation: Data Type

The observation implies that the automated refactoring systems should not be solely evaluated by the compilation correctness of the code and the test cases' passing results. The code behavior has been correctly preserved in the refactoring operation, but the actual goal of the refactoring operation may not be achieved correctly. The refactoring structure correctness should be explicitly evaluated by the defined criteria.

The difference also implies that it is not enough to just produce syntactically correct code in refactoring. The tests' passing primarily assures behavioral preservation, whereas the Extract Method refactoring needs more regarding the structure of the code. A possible solution is to incorporate some structure constraints to verify if the refactoring satisfies the requirement of the Extract Method.

6.2 Effect of Clone Complexity on Model Stability

In all the clone types, the same phenomenon is observed: the more complex the clones, the more difficult it is to refactor the code well. In the single LLM setting, Type-1 clones are easier to handle, Type-2 is worse, and Type-3 is the worst. The same phenomenon can be seen in all the models. This is not a syntax problem. Moreover, many of the code can still be compiled successfully and pass test cases. The real challenge is to determine the boundary of the extraction code in the Extract Method refactoring. In the case of Type-1 clones, the two code blocks are identical, so the Extract Method refactoring is easy to perform. However, in the case of Type-2 and Type-3 clones, the two code blocks are similar but not identical. The model has to determine what code should be included in the common helper method, what code should remain in the original methods. The differences inside the body of the clone code, especially for Type-3 clones, make it much easier to fail.

In fact, our multi-agent workflow can alleviate some of the above failures by adding structural checks and revisions. For example, if the initial attempt did not extract enough, over-extracted, or left partial duplication, the workflow can detect this and prompt for another attempt. Yet, the remaining gap between clone types indicates that this is still the problem. Even with feedback, the system cannot find the right boundary if the underlying structure is unclear.

For the future refactoring systems, boundary selection should be considered a first-class problem. There are two possible solutions to this: one is to incorporate some lightweight program analysis to provide candidate boundaries for refactoring, depending on data flow or statement dependencies, and then let the model refine within this constrained space. Another possible solution is to make the workflow more adaptive: for more difficult clones, especially for Type-3 clones, the system could ask for stronger verification, or request more reasoning, for instance, why this boundary was chosen. Otherwise, the success rate will probably continue to drop for more complex clones, even though the code will compile and pass the test.

6.3 Component Interaction and Dependency

The ablation study in RQ2 demonstrates that all components play an important role in the overall workflow performance. The full workflow has an 86.3% success rate. If RAG is removed, the success rate drops to 72.5%. If the usefulness checker is removed, it drops to 66.3%. If the feedback loop is removed, it drops to 68.3%. This demonstrates that it is not one but rather a combination of mechanisms that drive the overall performance of the system.

The magnitude of each drop also gives insight into how each component works. RAG primarily deals with the initial refactoring code. The retrieval guidance increases the likelihood that the model produces structurally correct refactoring attempts from the start. The usefulness checker has the largest effect since it eliminates refactorings that preserve behavioral correctness but fail to meet the structural requirements of Extract Method. The feedback loop also plays an important role in fixing boundary issues and incomplete extractions in the refactoring process.

This is confirmed by the ablation study, where we found that a single component or a single generation step is not sufficient for generating refactoring in a reliable way. Rather, we need multiple components that support different parts of the refactoring workflow.

In more detail, the retrieval guidance is useful for generating better refactoring code, and the structural validation is useful for ensuring that the refactoring is correct and meets the requirements of the Extract Method refactoring. The feedback loop is then useful for revising incorrect or incomplete refactoring attempts. This implies that more research should be conducted on the workflow level rather than on a single component such as code generation in the future.

In conclusion, the results of our study indicate that in order to generate refactoring in a reliable way, we need to support the generation, validation, and revision of refactoring, and systems that explicitly support all these mechanisms are likely to be more stable and reliable than systems that rely on better prompts or model sizes.

6.4 Usefulness Comparison: JDeodorant vs. Multi-Agent Workflow

RQ3 enables a direct comparison between JDeodorant and our multi-agent workflow under the same 60 clone cases. Deckard detected 41 cases (68.3%). Among these, JDeodorant reported 6 precondition violations and applied Extract Method to the remaining 35 cases. Out of these 35 cases, 34 were successfully refactored and considered useful, yielding a 97.1% success rate on the applicable subset.

However, this high percentage reflects performance only on cases that satisfy the tool's internal preconditions. When viewed over the entire dataset, JDeodorant does not attempt refactoring on undetected clones or on detected cases that violate its safety constraints. These preconditions are designed to ensure that behavior is preserved and that extraction does not break Java-specific data-flow or type constraints.

In contrast, the multi-agent workflow attempts refactoring for all 60 clone cases and achieves 85.8% usefulness overall. Although this percentage is lower than 97.1% on JDeodorant's applicable subset, it reflects end-to-end performance across a broader range of clone scenarios.

This comparison shows that there are two completely different strategies in automated refactoring systems. The JDeodorant refactoring tool has a conservative design, meaning that refactoring is done only if all conditions for correctness are formally satisfied. This results in a very high precision, but the number of possible refactoring tasks is limited. The multi-agent workflow has a completely different design strategy: it tries to perform a refactoring and then relies on the feedback from usefulness checks, compilations, and tests to correct wrong results.

This shows that there is a trade-off between precision and applicability in automated refactoring systems. The precision of rule-based refactoring systems is ensured by strict preconditions, but this is at the cost of applicability. The precision of LLM-based workflows is lower, and there are more possibilities for refactoring but there are still mechanisms needed to ensure structural correctness.

During manual inspection of the refactoring results produced by JDeodorant, we also noticed a stylistic difference between JDeodorant and the multi-agent workflow. In a few instances of Type-2 and Type-3 clones, JDeodorant uses lambda expressions when performing an Extract Method refactoring. This is particularly common when restructuring code using a functional interface is needed. However, none of the produced refactorings in the multi-agent workflow contained lambda expressions. The above is a reflection of how JDeodorant and the multi-agent workflow generate their refactoring results. JDeodorant relies on predefined refactoring rules that support language-specific code transformations. The LLM-based workflow, on the other hand, tends to produce simpler extractions that keep the original structure of the code.

The lack of lambda expressions in the LLM output does not necessarily imply incorrect transformation. In most cases, the methods still remove duplication and behave correctly. Instead, this implies that the LLM-based refactoring methods currently focus mostly on structural duplication removal, rather than more advanced language features.

There is an important implication of this for the development of refactoring tools in the future. Traditional refactoring tools implement detailed knowledge of refactoring at the language level, while LLM-based tools exploit more general capabilities for generating code. A potential way to make refactoring tools more robust in the future could be to exploit this combination of capabilities: for instance, LLM-generated candidates could be guided or post-processed using language-aware transformation rules so that the final refactorings not only remove duplication but also follow more advanced programming patterns.

6.5 Model-Specific Failure Patterns

The prior empirical work [35] found several common incorrect Extract Method refactoring patterns, such as incomplete refactoring, direct clone removal without introducing a shared helper method, and fragmentation of logic. In all these cases, a transformation is applied but the resulting code does not perfectly match the definition of the Extract Method refactoring. Although code duplication is reduced to a certain degree, a proper shared method is not created.

In our RQ2 ablation study, we also found another incorrect refactoring pattern. In 13 cases generated by DeepSeek v3.2, both cloned methods are returned unchanged. No helper method is created and no duplicated code is extracted. Unlike the above incorrect refactoring patterns, this one is not an incorrect transformation. It is actually a transformation where no code is changed.

This is why detection, compilation, and testing remain constant in ablation settings but usefulness and overall success are reduced. Compilation and testing check for behavioral correctness but cannot guarantee that duplication was actually removed. By definition of the Extract Method refactoring, a successful refactoring must both preserve behavior and introduce a shared abstraction. When the output is unchanged from the original code, a refactoring's structural objective is not satisfied, even though code is still correctly compilable and testable.

The absence of unchanged output in full multi-agent workflow is an important point. When retrieval guidance and structural feedback are both enabled, a transformation is always applied. This indicates that structural validation is important not just for refining incorrect refactoring but also for eliminating cases where no refactoring is produced.

These observations also highlight an evaluation concern for automatic refactoring tools. Indeed, behavioral tests alone are not sufficient to distinguish between a correct refactoring and no operational output. Instead, it is advised that an evaluation framework should be able to detect if a correct transformation has been applied or not. Moreover, workflow-level features such as retrieval guidance and iterative validation can be used to ensure that meaningful structural changes are made by the generated output instead of no operation.

6.6 Consideration of Type-4 Clones

While our experiments are centered on Type-1, Type-2, and Type-3 clones, a brief discussion on how the workflow might be extended to Type-4 clones is worthwhile.

Type-4 clones are semantically similar, but syntactically different from each other. Because of this, structural or token-based similarity is generally not sufficient to detect Type-4 clones.

While our experimental results in RQ1 and RQ2 indicate that structural enforcement is generally insufficient, especially if syntax is significantly different in the case of Type-4 clones, the issue is no longer how to find duplicated structure, but rather how to find duplicated behavior. This might be achieved by employing a number of semantic analysis strategies, such as program dependence graphs, program execution graphs, or embedding-based similarity analysis, in order to find duplicated behavior in the code.

In addition, refactoring Type-4 clones would require a higher level of behavioral guarantees, as the extraction boundary might no longer be clear, and behavioral preservation is now more important. This implies that test coverage must now be sufficient to cover the shared behavior, and possibly additional constraints must now be imposed on the generation process, such as data dependency preservation and side effect preservation from both code fragments.

Moreover, the usefulness checker would also have to be adapted. Currently, our usefulness checker is based on structural information. In the case of Type-4 clones, it is possible that structural similarity is already low. Therefore, it could be necessary to extend the usefulness evaluation to semantic consolidation instead of token reduction. This could be achieved by checking if both original code fragments refer to a common abstraction that expresses the same steps, even if these steps are represented by different internal statements.

We do not claim that our current workflow is able to deal with Type-4 clones directly. However, we hope that the results of this study can be used as a basis for design decisions for an extension of existing clone detection tools to Type-4 clones. This would require a more powerful semantic analysis and additional evaluation tools, which is left for future work.

6.7 Code Quality Considerations

Our evaluation criteria include structural correctness, behavioral preservation, and usefulness according to the definition of the Extract Method technique. However, we are not measuring other code quality attributes such as complexity, cohesion, coupling, readability, and maintainability.

It is possible that two different refactorings may pass all tests and be correct according to structural criteria but differ in code quality attributes. For example, a function may be extracted and be correct according to our validation criteria but still be a bad function because it has too many parameters, it has reduced cohesion, or it has increased coupling with other classes.

As a possible extension of this work, it could be interesting to include existing code quality analysis tools to measure code quality after the refactoring. For example, tools such as Understand or other software metrics analyzers could be used to measure complexity, dependency graphs, and cohesion before and after the refactoring process.

In addition, quality optimization itself might be a role that is added to the workflow explicitly, in the form of a quality-aware agent. The agent might analyze changes in metrics and provide feedback on how to improve the code further. It is worth noting that this agent would not replace structural or behavioral validation, but rather complement it, focusing on design improvements.

While we are not claiming that the current workflow improves overall quality better than duplication reduction, we are establishing a base that is centered around structural validity and correctness, and that improving the workflow in a multi-dimensional quality optimization is a significant area for future research.

6.8 Expanding Clone Detection and Refactoring Across Files

One limitation with our current workflow is that it only looks for clones within the current file. Since the tool is triggered when a developer pastes code, it focuses its analysis on the file where the paste occurs. However, in real software projects, duplication can often occur across multiple files rather than within a single file.

Addressing this limitation is important because supporting cross-file clone detection would significantly increase the impact of our tool. Clones across different files can be even harder for developers to detect, increasing maintenance issues and inconsistencies over time. Clones within the same file are usually easy to spot, but when code is duplicated across files, there is no easy way for a developer

to detect them without explicitly searching. The larger and more complicated the project becomes, the more valuable this becomes as the chances of duplicating logic across different files only increase with size and complexity. This is the kind of problem our tool aims to prevent, so extending it to detect clones across files would enable it to catch duplications that, in most cases, would otherwise go unaddressed.

The foundation established in this study provides a good starting point for this extension. Future work could build on this workflow to support cross-file detection and refactoring, and evaluate whether it retains the same improvements in usefulness and correctness at a larger scale. Although this extension was not included in the current version of the tool, it represents an important direction for future work in making the workflow more practical for real-world development.

7 THREATS TO VALIDITY

Internal validity. One potential threat comes from how usefulness is evaluated. In this study, usefulness is approximated using rule-based structural checks, such as clone similarity reduction and PSI-level evidence of Extract Method refactoring. These checks are motivated by failure patterns observed in the empirical study [35]. However, they still reflect specific assumptions about what constitutes a meaningful Extract Method refactoring. It is possible that some refactorings marked as not useful by our checker would be acceptable to developers in certain contexts.

Another internal threat comes from the test coverage of the EvoSuite tests. In the proposed workflow, testing is used as feedback for the preservation of behavior after refactoring. However, the tests, although automated, may not completely cover the cloned methods. Out of 60 clone cases, 35 of them have test coverage that directly tests the cloned methods. For the remaining cases where the cloned methods are not tested, we manually check the code after refactoring to ensure that behavior is preserved. Although this reduces the threat of semantic changes, the test coverage may influence the validity of the testing feedback used for evaluation.

External validity. The evaluation of the workflow is restricted to Java-based projects, while only one type of refactoring Extract Method is used for evaluation purposes. The outcomes of the evaluation may not be applicable to other programming languages, as well as other types of refactoring operations. Besides, the workflow is initiated by paste operations, while clones within one file are used for analysis purposes. This approach is based on the interactive IDE scenario, which may not account for scenarios where clones emerge over time across files.

Another threat to the study's external validity is associated with the creation of the clones used for evaluation purposes. The 60 clone instances used for evaluation purposes are manually constructed to resemble real-world scenarios of Type-1, Type-2, and Type-3 clones within the context of the JHotDraw application, rather than being directly mined from large-scale repositories. Although we design these cases to reflect different clone types, they may not fully capture the diversity, contextual complexity of real-world clones.

The concept of usefulness adopted in this study is related to the concept of Extract Method refactoring. Different teams or projects can have different criteria for determining the usefulness of the refactoring. For instance, different people can have different criteria for the refactoring. To minimize the subjective nature of the refactoring, the adopted criteria for determining the usefulness of the refactoring are based on the failure modes or refactoring patterns identified in the empirical study [35], such as incomplete refactoring, excessive refactoring, or logic fragmentation. However, it is accepted that different definitions of the concept of usefulness can exist, and the use of different criteria can yield different results.

8 CONCLUSION

This paper studies whether a structured multi-agent workflow can improve the reliability of LLM-based clone refactoring. We focus on Extract Method refactoring triggered by copy–paste actions in the IDE.

Our results show that decomposing the task into specialized agents significantly improves the rate of structurally valid Extract Method refactorings. Compared with a single-LLM setting, the multi-agent workflow increases overall accepted refactoring success from 70.4% to 85.8%. The main improvement comes from reducing structurally incorrect or incomplete extractions. The usefulness checker plays a critical role by filtering out refactorings that compile and pass tests but do not truly remove duplication. Retrieval-augmented generation helps the model generate a first candidate that better follows the Extract Method structure, while the feedback loop helps correct borderline failures.

The ablation study further shows that the components do not work independently. Removing RAG, usefulness checking, or iterative feedback each leads to a noticeable drop in performance. When multiple components are removed together, the performance drops more sharply. We also observe that weaker models may fail to apply any meaningful refactoring once structural guidance is removed, even if compilation and tests succeed. This highlights the importance of structural validation beyond behavioral checking.

Overall, our findings suggest that reliable LLM-based refactoring requires more than prompt engineering. Structural constraints, behavioral validation, and component coordination are necessary to ensure both correctness and usefulness in interactive development settings. Moreover, the tool is model-agnostic and can be extended to support additional LLMs as new models become available, making the framework adaptable to future advances in large language models.

REFERENCES

[1] 2026. Replication Package for Multi-agent LLM-based Framework for Code Clone Refactoring. https://byte-shadow37.github.io/multi_agent_replication_package/. Accessed: 2026-03-05.

[2] Ajmain I Alam, Palash R Roy, Farouq Al-Omari, Chanchal K Roy, Banani Roy, and Kevin A Schneider. 2023. GPTCloneBench: A comprehensive benchmark of semantic clones and cross-language clones using GPT-3 model and SemanticCloneBench. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 1–13.

[3] Eman Abdullah AlOmar, Jacob Ashkenas, Robert Feliciano, Matthew Angelakos, Dimitrios Haralamppopoulos, Xing Qian, Mohamed Wiem Mkaouer, and Ali Ouni. 2025. AntiCopyPaster 3.0: Just-in-Time Clone Refactoring. *ACM Transactions on Software Engineering and Methodology* (2025).

[4] Eman Abdullah AlOmar, Anton Ivanov, Zarina Kurbatova, Yaroslav Golubev, Mohamed Wiem Mkaouer, Ali Ouni, Timofey Bryksin, Le Nguyen, Amit Kini, and Aditya Thakur. 2022. AntiCopyPaster: extracting code duplicates as soon as they are introduced in the IDE. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–4.

[5] Eman Abdullah AlOmar, Anton Ivanov, Zarina Kurbatova, Yaroslav Golubev, Mohamed Wiem Mkaouer, Ali Ouni, Timofey Bryksin, Le Nguyen, Amit Kini, and Aditya Thakur. 2023. Just-in-time code duplicates extraction. *Information and Software Technology* 158 (2023), 107169.

[6] Eman Abdullah AlOmar, Benjamin Knobloch, Thomas Kain, Christopher Kalish, Mohamed Wiem Mkaouer, and Ali Ouni. 2024. AntiCopyPaster 2.0: Whitebox just-in-time code duplicates extraction. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 84–88.

[7] Eman Abdullah AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. 2024. Behind the intent of extract method refactoring: A systematic literature review. *IEEE Transactions on Software Engineering* 50, 4 (2024), 668–694.

[8] Eman Abdullah AlOmar, Luo Xu, Sofia Martinez, Anthony Peruma, Mohamed Wiem Mkaouer, Christian D Newman, and Ali Ouni. 2025. ChatGPT for code refactoring: Analyzing topics, interaction, and effective prompts. In *2025 IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*. IEEE, 389–398.

[9] Abhiram Bellur, Mohammed Raihan Ullah, Fraol Batole, Mohit Kansara, Masaharu Morimoto, Kai Ishikawa, Haifeng Chen, Yaroslav Zharov, Timofey Bryksin, Tien N Nguyen, et al. 2026. Multi-Agent Coordinated Rename Refactoring. *arXiv preprint arXiv:2601.00482* (2026).

[10] Xiliang Chen, Alice Yuchen Wang, and Ewan Tempero. 2014. A replication and reproduction of code clone detection studies. In *Proceedings of the Thirty-Seventh Australasian Computer Science Conference-Volume 147*. 105–114.

[11] William G Cochran. 1950. The comparison of percentages in matched samples. *Biometrika* 37, 3/4 (1950), 256–266.

[12] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2024. An empirical study on the code refactoring capability of large language models. *arXiv preprint arXiv:2411.02320* (2024).

[13] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2025. LLM-Driven Code Refactoring: Opportunities and Limitations. In *2025 IEEE/ACM Second IDE Workshop (IDE)*. IEEE, 32–36.

[14] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 758–759.

[15] Kayla DePalma, Izabel Miminoshvili, Chiara Henselder, Kate Moss, and Eman Abdullah AlOmar. 2024. Exploring ChatGPT's code refactoring capabilities: An empirical study. *Expert Systems with Applications* 249 (2024), 123602.

[16] Shihan Dou, Junjie Shan, Haoxiang Jia, Wenhao Deng, Zhiheng Xi, Wei He, Yueming Wu, Tao Gui, Yang Liu, and Xuanjing Huang. 2023. Towards understanding the capability of large language models on code clone detection: A survey. *arXiv preprint arXiv:2308.01191* (2023).

[17] Olive Jean Dunn. 1961. Multiple comparisons among means. *Journal of the American statistical association* 56, 293 (1961), 52–64.

[18] Martin Fowler. 2018. *Refactoring: improving the design of existing code*. Addison-Wesley Professional.

[19] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.

[20] Bowen Jiang, Mitchell Ruffolo, Aaditya Kulkarni, Teia Canini, and Eman Abdullah AlOmar. 2026. A systematic mapping study on the research landscape of LLM-based code clone detection. *Information and Software Technology* (2026), 108096.

[21] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. 2007. Deckard: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 96–105.

[22] Mohamad Khajezade, Jie JW Wu, Fatemeh Hendijani Fard, Gema Rodríguez-Pérez, and Mohamed Sami Shehata. 2024. Investigating the efficacy of large language models for code clone detection. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 161–165.

[23] Miryung Kim, Vibha Sazawal, David Notkin, and Gail Murphy. 2005. An empirical study of code clone genealogies. In *Proceedings of the 10th European software engineering conference held jointly with 13th ACM SIGSOFT international symposium on Foundations of software engineering*. 187–196.

[24] Egambaram Kodhai and Selvadurai Kanmani. 2014. Method-level code clone detection through LWH (Light Weight Hybrid) approach. *Journal of Software Engineering Research and Development* 2 (2014), 1–29.

[25] Yun Lin, Zhenchang Xing, Yinxing Xue, Yang Liu, Xin Peng, Jun Sun, and Wenyun Zhao. 2014. Detecting differences across multiple instances of code clones. In *Proceedings of the 36th International Conference on Software Engineering*. 164–174.

[26] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2024. An empirical study on the potential of llms in automated software refactoring. *arXiv preprint arXiv:2411.04444* (2024).

[27] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah Alomar. 2025. Software Refactoring Research with Large Language Models: A Systematic Literature Review. *Journal of Systems and Software* (2025), 112762.

[28] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.

[29] Emerson Murphy-Hill, Chris Parnin, and Andrew P Black. 2011. How we refactor, and how we know it. *IEEE Transactions on Software Engineering* 38, 1 (2011), 5–18.

[30] Suphakit Niwattanakul, Jatsada Singthongchai, Ekkachai Naenudorn, and Supachanun Wanapu. 2013. Using of Jaccard coefficient for keywords similarity. In *Proceedings of the international multiconference of engineers and computer scientists*, Vol. 1. 380–384.

[31] Mel Ó Cinnéide, Laurence Tratt, Mark Harman, Steve Counsell, and Iman Hemati Moghadam. 2012. Experimental assessment of software metrics using automated refactoring. In *Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement*. 49–58.

[32] Khouloud Oueslati, Maxime Lamothe, and Foutse Khomh. 2025. RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring. *arXiv preprint arXiv:2511.03153* (2025).

[33] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Next-generation refactoring: Combining llm insights and ide capabilities for extract method. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 275–287.

[34] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Andrey Sokolov, Timofey Bryksin, and Danny Dig. 2024. Em-assist: Safe automated extractmethod refactoring with llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 582–586.

[35] Xing Qian and Eman Abdullah Alomar. 2025. Can LLMs Identify and Refactor Code Clones? An Empirical Study. *Journal of Systems and Software* (2025), 112717.

[36] Chaiyong Ragkhitwetsagul, Jens Krinke, and David Clark. 2018. A comparison of code similarity analysers. *Empirical Software Engineering* 23, 4 (2018), 2464–2519.

[37] Chanchal K Roy, James R Cordy, and Rainer Koschke. 2009. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of computer programming* 74, 7 (2009), 470–495.

[38] Shahbaz Siddeeq, Muhammad Waseem, Zeeshan Rasheed, Md Mahade Hasan, Jussi Rasku, Mika Saari, Henri Terho, Kalle Mäkelä, Kai-Kristian Kemell, and Pekka Abrahamsson. 2025. LLM-Based Multi-agent System for Intelligent Refactoring of Haskell Code. In *International Conference on Product-Focused Software Process Improvement*. Springer, 408–418.

[39] Jeffrey Svajlenko, Judith F Islam, Iman Keivanloo, Chanchal K Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *2014 IEEE international conference on software maintenance and evolution*. IEEE, 476–480.

[40] Robert Tairas, Jeff Gray, and Ira Baxter. 2006. Visualization of clone detection results. In *Proceedings of the 2006 OOPSLA workshop on eclipse technology eXchange*. 50–54.

[41] Nikolaos Tsantalis, Davood Mazinanian, and Giri Panamoottil Krishnan. 2015. Assessing the refactorability of software clones. *IEEE Transactions on Software Engineering* 41, 11 (2015), 1055–1090.

[42] Yisen Xu, Feng Lin, Jinqiu Yang, Nikolaos Tsantalis, et al. 2025. Mantra: Enhancing automated method-level refactoring with contextual rag and multi-agent llm collaboration. *arXiv preprint arXiv:2503.14340* (2025).

[43] Zixian Zhang and Takfarinas Saber. 2024. Assessing the code clone detection capability of large language models. In *2024 4th International Conference on Code Quality (ICCQ)*. IEEE, 75–83.